From Diffusion to Flow: Principles and Scientific Applications of Modern Generative Models

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

November 11, 2025

Feynman's words

What I cannot create, I do not understand.

Outline

Outline

Generate samples from given distribution

Given a distribution $p(x)$, how to sample from the distribution?

- Uniform distribution & Gaussian distribution: easy sampling
- Mixture Gaussian: determine which Gaussian to sample & sample from that Gaussian
- A general sample function?
  - Langevin Markov Chain Monte Carlo sampling (Langevin MCMC)
  - Stein Variational Gradient Descent (SVGD)
  - ...

Score function is all you need?

A typical process of generating new samples from modeling the data distribution

1. To find the probability function $p(x)$, model $p(x;\theta)$.
2. Normalization distribution $p(x;\theta) = \frac{1}{C(\theta)} e^{-E(x;\theta)}$ with $C(\theta) \equiv \int_x e^{-E(x;\theta)} dx$.
3. Generating samples from above sampling methods.

Recall the Langevin Markov Chain Monte Carlo sampling

$$x_t = x_{t-1} + \frac{\Delta t}{2} \nabla_x \log p(x_{t-1};\theta) + \sqrt{\Delta t}\epsilon, \quad \epsilon \sim N(0,1). \tag{1}$$

Here $\nabla_x \log p(x_{t-1};\theta) = -\nabla_x E(x;\theta)$ is free of the normalization $C(\theta)$.
Define the score function

$$S(x;\theta) \equiv \nabla \log p(x;\theta). \tag{2}$$

More about Langevin MCMC and the score function

Continuous form of the Langevin MCMC

- $\Delta t \to 0$

$$dx = \frac{1}{2}\nabla_x \log p(x)dt + dB_t, \tag{3}$$

  where dB_t is a Brownian motion.
- Given $p(x) = \frac{1}{C}e^{-E}$, we have

$$dx = -\frac{1}{2}\nabla_x E dt + dB_t \tag{4}$$

Langevin MCMC is overdamped Langevin dynamics

- Langevin dynamics (MD)

$$\ddot{x} = -\nabla_x E - \gamma\dot{x} + dB_t, \tag{5}$$

- Given $\gamma\dot{x} \gg \ddot{x}$, we have the overdamped Langevin equation

$$dx = -\frac{1}{\gamma}\nabla_x E + \frac{1}{\gamma}dB_t. \tag{6}$$

Outline

Matching the score by neural networks

Recall the definition

$$S \equiv \nabla_x \log p(x), \tag{7}$$

The key is to matching the score by a neural network

$$J_{ESM} = \frac{1}{2}\mathbb{E}_p \left[\| s(x;\theta) - \frac{\partial \log p(x)}{\partial x} \|^2 \right], \tag{8}$$

where J_{ESM} is called Explicit Score Matching loss. However, it is not tractable to compute $\frac{\partial \log p(x)}{\partial x}$ from data.

Where should we go?

Implicit Score Matching (ISM) loss

Implicit Score Matching loss (Aapo, 2005) was developed to make a tractable score matching

$$J_{ISM} \equiv \mathbb{E}_p \left[\frac{1}{2} \|s(x;\theta)\|^2 + \nabla \cdot s(x;\theta) \right] = J_{ESM} - C. \tag{9}$$

Denoising Score Matching (DSM) loss

However, the ISM score is not very stable to optimize, with two reasons

1. Expectation is performed on the whole distribution;
2. The loss is negative decreasing to $-C$ with a great quantity, e.g. $-1e5$.

Denoising Score Matching (DSM) loss (Vincent, 2011) is developed to solve this problem

$$J_{DSM} = \frac{1}{2}\mathbb{E}_{p(x,\tilde{x})}\left[\|s(\tilde{x};\theta) - \nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2\right]. \tag{10}$$

And we can prove that $J_{DSM} = J_{ESM} + C$.

Key Insight: Denoising Score Matching

Denoising Score Matching (DSM) loss (Vincent, 2011) solves the optimization stability problem:

$$J_{DSM} = \frac{1}{2}\mathbb{E}_{p(x,\tilde{x})}\left[\|s(\tilde{x};\theta) - \nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2\right]. \tag{11}$$

Key insight: $J_{DSM} = J_{ESM} + C$ where C is a constant, making it tractable without computing the true score.

Challenge: When $\tilde{x} \to x$ (low noise), $J_{DSM} \to \infty$. This motivates the need for multi-scale noise schedules in diffusion models.

Outline

Revisit the adding noise from x to $\tilde{x}$

Gaussian transition kernel for adding noise from x to $\tilde{x}$

$$\tilde{x} = \alpha x + \sigma\epsilon, \tag{12}$$

which equivalent to the Gaussian transition kernel

$$p(\tilde{x}|x) = N(\tilde{x}; \alpha x, \sigma^2), \tag{13}$$

where $\epsilon \sim N(0, 1)$ and

$$N(\tilde{x}; \alpha x, \sigma^2) = Ce^{-\frac{\|\tilde{x}-\alpha x\|^2}{2\sigma^2}}. \tag{14}$$

And

$$\epsilon = \frac{\tilde{x} - \alpha x}{\sigma}. \tag{15}$$

Questions arise: how can we get x given $\tilde{x}$?

Tweedie's Formula

Tweedie's Formula (Bayes statistics) states (Robbins, 1956)

Theorem

Given random variables x, y, $y \sim N(x, \sigma^2 I)$, i.e., $p(y|x) = N(x, \sigma^2)$, the expectation of x could be given by

$$\mathbb{E}[x|y] = y + \sigma^2 \nabla \log p(y). \tag{16}$$

Let

$$x \leftarrow \alpha x, \quad y \leftarrow \tilde{x}, \tag{17}$$

we have

$$s \equiv \nabla_{\tilde{x}} \log p(\tilde{x}) = -\frac{\tilde{x} - \alpha \mathbb{E}[x|\tilde{x}]}{\sigma^2} = -\frac{1}{\sigma}\mathbb{E}[\frac{\tilde{x} - \alpha x}{\sigma}|\tilde{x}] = -\frac{\mathbb{E}[\epsilon|\tilde{x}]}{\sigma}, \tag{18}$$

where we have used the fact $\epsilon = \frac{\tilde{x} - \alpha x}{\sigma}$.

Another insight: from the definition of the score S

Recall that adding noise by $\tilde{x} = \alpha x + \sigma\epsilon$,

$$p(\tilde{x}|x) = Ce^{-\frac{\|\tilde{x}-\alpha x\|^2}{2\sigma^2}}. \tag{19}$$

Hence

$$s(\tilde{x}) = \nabla_{\tilde{x}} \log p(\tilde{x}) = \frac{\nabla_{\tilde{x}} p(\tilde{x})}{p(\tilde{x})} = \frac{\int_x \nabla_{\tilde{x}} p(\tilde{x}|x)p(x)dx}{p(\tilde{x})}, \tag{20}$$

From Eq. (19), we have

$$s(\tilde{x}) = \frac{\int_x \nabla_{\tilde{x}} \log p(\tilde{x}|x)p(\tilde{x}|x)p(x)dx}{p(\tilde{x})} = \int_x \nabla_{\tilde{x}} \log p(\tilde{x}|x)p(x|\tilde{x})dx, \tag{21}$$

leading to

$$s(\tilde{x}) = \mathbb{E}_x[\nabla_{\tilde{x}} \log p(\tilde{x}|x)|\tilde{x}] = \mathbb{E}_x[-\frac{\tilde{x}-\alpha x}{\sigma^2}|\tilde{x}] = -\frac{\tilde{x}-\alpha\mathbb{E}[x|\tilde{x}]}{\sigma^2}. \tag{22}$$

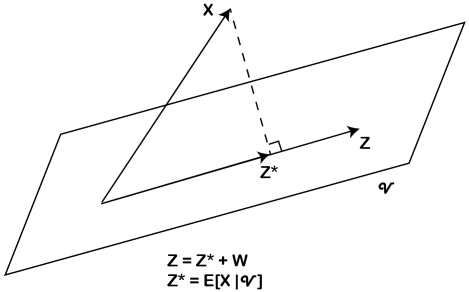
Theoretical Support: Conditional Expectation

Theorem

Let X be an integrable random variable. Then for each σ-algebra $\mathcal{V}$ and $Z \in \mathcal{V}$, $Z^* = \mathbb{E}(X|\mathcal{V})$ solves the least square problem

$$argmin_{Z \in \mathcal{V}} \|Z - X\|,$$

where $\|Z\| = \left(\int Z^2 dP\right)^{\frac{1}{2}}$.



Setting

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space and $L^2 := L^2(\Omega, \mathcal{F}, \mathbb{P})$ with $\langle U, V \rangle = \mathbb{E}[UV]$. Fix a sub-$\sigma$-field $\mathcal{G} \subset \mathcal{F}$ and denote the closed subspace

$$\mathcal{V} := L^2(\mathcal{G}) = \{Z \in L^2 : Z \text{ is } \mathcal{G}\text{-measurable}\}.$$

Claim. For $X \in L^2$,

$$Z^* := \mathbb{E}[X \mid \mathcal{G}] \in \mathcal{V} \quad \text{and} \quad Z^* = \arg\min_{Z \in \mathcal{V}} \mathbb{E}[(Z - X)^2].$$

Proof of the claim

For any $Z \in \mathcal{V}$,

$$\langle X - Z^*, Z \rangle = \mathbb{E}[(X - Z^*)Z] = \mathbb{E}(\mathbb{E}[(X - Z^*)Z \mid \mathcal{G}]) = \mathbb{E}(Z\,\mathbb{E}[X - Z^* \mid \mathcal{G}]) = 0.$$

Hence $X - Z^* \perp \mathcal{V}$.

Let any $Z \in \mathcal{V}$ be written as $Z = Z^* + W$ with $W \in \mathcal{V}$. Then

$$\|X - Z\|_2^2 = \|X - (Z^* + W)\|_2^2 = \|X - Z^*\|_2^2 + \|W\|_2^2 \geq \|X - Z^*\|_2^2,$$

because $\langle X - Z^*, W \rangle = 0$. Equality $\Leftrightarrow W = 0 \Leftrightarrow Z = Z^*$, proving uniqueness and the minimizer claim.

Revisit several models

1. Score model
$$Loss_{score} = \|s_\theta(\tilde{x}) - \frac{\epsilon}{\sigma}\|^2; \tag{23}$$
2. X prediction model (denoising model, $x0$ model)
$$Loss_{x_0} = \|x_\theta(\tilde{x}) - x\|^2, \tag{24}$$
with
$$s(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta^*}(\tilde{x})}{\sigma^2} \tag{25}$$
where $x_\theta(\tilde{x}) \to x_{\theta^*}(\tilde{x}) \equiv \mathbb{E}[x|\tilde{x}]$;
3. ϵ prediction model
$$Loss_\epsilon = \|\epsilon_\theta(\tilde{x}) - \epsilon\|^2, \tag{26}$$
with
$$s(\tilde{x}) = -\frac{\epsilon_{\theta^*}(\tilde{x})}{\sigma}, \tag{27}$$
where $\epsilon_\theta(\tilde{x}) \to \epsilon_{\theta^*}(\tilde{x}) \equiv \mathbb{E}[\epsilon|\tilde{x}]$.

The consistency of the three modeling

As stated before, we have the following connection in the optimal (θ^*) case

$$s_{\theta^*}(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta^*}(\tilde{x})}{\sigma^2} = -\frac{\epsilon_{\theta^*}(\tilde{x})}{\sigma}, \tag{28}$$

by the above conditional expectation.
Hence, in the real modeling of designing the loss, we use the connection of three models without reaching the optimal parameter

$$s_{\theta}(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta}(\tilde{x})}{\sigma^2} = -\frac{\epsilon_{\theta}(\tilde{x})}{\sigma}, \tag{29}$$

and substitute each other form to the loss definition above, we can recover all of the losses given by different models.

Outline

Key Identity: Jacobian Switch (chain rule between $\mathbf{x}_0$ and $\mathbf{x}_t$)

The Gaussian transition density is

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t;\ \alpha_t\,\mathbf{x}_0,\ \sigma_t^2\mathbf{I}\big) \propto \exp\Big(-\tfrac{\|\mathbf{x}_t-\alpha_t\,\mathbf{x}_0\|^2}{2\sigma_t^2}\Big).$$

A basic but crucial identity:

$$\nabla_{\mathbf{x}_t} q(\mathbf{x}_t \mid \mathbf{x}_0) = -\frac{1}{\alpha_t}\,\nabla_{\mathbf{x}_0} q(\mathbf{x}_t \mid \mathbf{x}_0), \qquad (\alpha_t > 0).$$

Proof sketch. Differentiating the exponent w.r.t. $\mathbf{x}_t$ and w.r.t. $\mathbf{x}_0$ shows $\nabla_{\mathbf{x}_t}\|\mathbf{x}_t - \alpha_t\mathbf{x}_0\|^2 = 2(\mathbf{x}_t - \alpha_t\mathbf{x}_0)$, $\nabla_{\mathbf{x}_0}\|\mathbf{x}_t - \alpha_t\mathbf{x}_0\|^2 = -2\alpha_t(\mathbf{x}_t - \alpha_t\mathbf{x}_0)$, thus the factor $-1/\alpha_t$.

Theorem 1: Score as Conditional Force Expectation

Statement. With $p(\mathbf{x}_0) \propto e^{-E(\mathbf{x}_0)/k_B T}$ and $\mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon}$,

$$\boxed{\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0|\mathbf{x}_t}\left[\frac{\mathbf{F}(\mathbf{x}_0)}{k_B T}\right],} \qquad \mathbf{F}(\mathbf{x}_0) = -\nabla_{\mathbf{x}_0} E(\mathbf{x}_0).$$

Derivation.

$$\begin{aligned}\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) &= \frac{\nabla_{\mathbf{x}_t} \int p(\mathbf{x}_0)\, q(\mathbf{x}_t \mid \mathbf{x}_0)\, \mathrm{d}\mathbf{x}_0}{p(\mathbf{x}_t)} \\ &= \frac{1}{p(\mathbf{x}_t)} \int p(\mathbf{x}_0)\, \nabla_{\mathbf{x}_t} q(\mathbf{x}_t \mid \mathbf{x}_0)\, \mathrm{d}\mathbf{x}_0\end{aligned}$$

Theorem 1: Derivation (Cont)

Proof (Cont).

Using the Jacobian switch identity $(*)$:

$$\begin{aligned}
&\overset{(*)}{=} \frac{1}{p(\mathbf{x}_t)} \int p(\mathbf{x}_0) \Big(-\frac{1}{\alpha_t} \Big) \nabla_{\mathbf{x}_0} q(\mathbf{x}_t \mid \mathbf{x}_0) \, \mathrm{d}\mathbf{x}_0 \\
&= -\frac{1}{\alpha_t} \frac{1}{p(\mathbf{x}_t)} \int q(\mathbf{x}_t \mid \mathbf{x}_0) \nabla_{\mathbf{x}_0} p(\mathbf{x}_0) \, \mathrm{d}\mathbf{x}_0
\end{aligned}$$

Theorem 1: Derivation (Final)

Proof (Cont).

$$= -\frac{1}{\alpha_t} \int \underbrace{\frac{p(\mathbf{x}_0) q(\mathbf{x}_t \mid \mathbf{x}_0)}{p(\mathbf{x}_t)}}_{p(\mathbf{x}_0 \mid \mathbf{x}_t)} \nabla_{\mathbf{x}_0} \log p(\mathbf{x}_0) \, \mathrm{d}\mathbf{x}_0$$

$$= \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0 \mid \mathbf{x}_t} \left[\frac{-\nabla_{\mathbf{x}_0} E(\mathbf{x}_0)}{k_B T} \right] = \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0 \mid \mathbf{x}_t} \left[\frac{\mathbf{F}(\mathbf{x}_0)}{k_B T} \right]$$

Experiments on first 1, 000 frames for LJ data

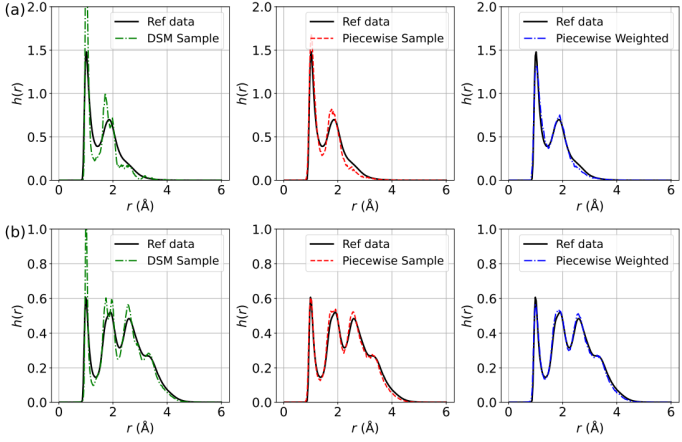


Figure 1: The distribution of interatomic distances $h(r)$ for LJ potential using biased training data with a sample size of 500. (a) Comparison of $h(r)$ for DSM, Piecewise, and Piecewise Weighted losses (from left to right) against reference data for LJ-13; (b) Comparison of $h(r)$ for DSM, Piecewise, and Piecewise Weighted losses for LJ-55.

Experiments on first 5, 000 frames for MD reference trajectories

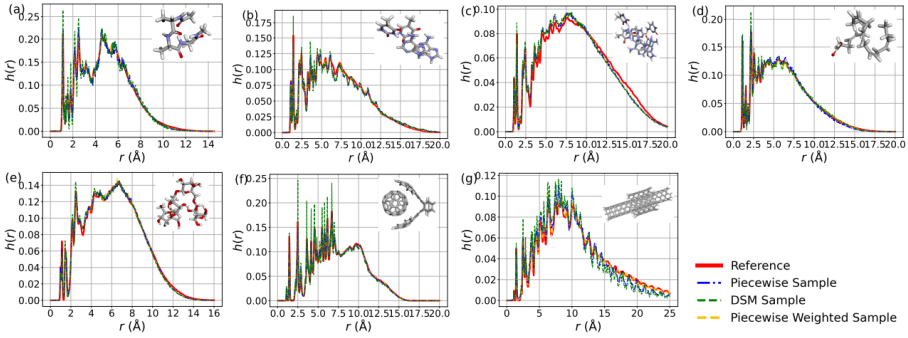


Figure 3: The distribution of interatomic distances (r(Å)) for (a) Ac-Ala3-NHMe, (b) DNA base pair (AT-AT), (c) DNA base pair (AT-AT-CG-CG), (d) Docosahexaenoic acid, (e) Stachyose, (f) Buckyball catcher, (g) Dw nanotube in MD22 dataset. The insets display the ball-and-stick representations of these molecules.

Outline

From Theory to Practice

Now that we have established the theoretical foundations of score matching and diffusion processes, let's explore how these concepts are implemented in practice through several key approaches:

- **SMLD** (Score Matching Langevin Dynamics)
- **DDPM** (Denoising Diffusion Probabilistic Models)
- **Continuous formulations** and their advantages

Each approach offers different insights into the practical implementation of diffusion-based generative models.

SMLD

Score Matching Langevin Dynamic (SMLD) (Song, 2019) adding noise in a following manner

$$p(\tilde{x}_i|x) = N(\tilde{x}_i; x, \sigma_i^2) \equiv Ce^{-\frac{\|\tilde{x}_i - x\|^2}{2\sigma_i^2}}, \tag{30}$$

where a geometric sequence $\sigma_1 > \sigma_2 > \cdots > \sigma_T \approx 0$ is given to add noise with different level. In a random variable view,

$$\tilde{x}_i = x + \sigma_i \epsilon \tag{31}$$

for different σ_i to get different noised random variable $\tilde{x}_i$.
Training object J_{DSM} is given by

$$J_{DSM} = \frac{1}{2}\mathbb{E}_{\sigma_i}\mathbb{E}_{p(x)}\mathbb{E}_{p(\tilde{x}_i|x)}\left[\|s_\theta(\tilde{x}_i, \sigma_i) + \frac{\tilde{x}_i - x}{\sigma_i^2}\|^2\right]. \tag{32}$$

Anneled Langevin MCMC for sampling

$$x_t = x_{t-1} + \frac{\Delta t}{2} S_\theta(x_{t-1}) + \sqrt{\Delta t}\epsilon, \quad \epsilon \sim N(0, 1). \tag{33}$$

DDPM

denoising diffusion probabilistic models (DDPM) (Ho, 2020)

1. Derived from the Evidence Lower BOund (ELBO), Not score matching.
2. First provide adding noise and denoise in the forward and reverse process.

The main procedure

1. Adding noise
$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon; \tag{34}$$
2. Transition kernel
$$p(x_t|x_0) = N(x_t; \alpha_t x_0, \sigma_t^2), \tag{35}$$
where $\alpha_t = \prod_{i=1}^{t}\sqrt{1-\beta_i}$ and $\sigma_t^2 = 1-\alpha_t^2$.

DDPM

The main procedure (Continued)

1. Adding noise
$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon; \tag{36}$$
2. Transition kernel
$$p(x_t|x_0) = N(x_t; \alpha_t x_0, \sigma_t^2), \text{ i.e., } x_t = \alpha_t x_0 + \sigma_t \epsilon, \tag{37}$$
where $\alpha_t = \prod_{i=1}^{t}\sqrt{1-\beta_i}$ and $\sigma_t^2 = 1-\alpha_t^2$.
3. Training Loss
$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{p(x)}\mathbb{E}_{p(x_t|x)}\left[\|\epsilon_\theta(x_t,t)-\epsilon\|^2\right], \tag{38}$$
where we have used the approximation $x_0 = \frac{x_t-\sigma_t\epsilon}{\alpha_t}$.
4. Sampling
$$x_{t-1} = \frac{1}{\sqrt{1-\beta_t}}(x_t + \beta_t s_\theta(x_t,t)) + \sqrt{\beta_t}\epsilon_t, \quad t = T, T-1, \cdots, 1. \tag{39}$$

Outline

The continuous version of SMLD

Recall that

$$x_t = x_0 + \sigma_t \epsilon, \tag{40}$$

we can prove that

$$x_t = x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\epsilon. \tag{41}$$

Proof.

We can prove the formula by a Recurrence

$$\begin{aligned} x_t &= x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\epsilon \\ &= x_{t-2} + \sqrt{\sigma_{t-1}^2 - \sigma_{t-2}^2}\epsilon + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\epsilon \\ &= x_{t-2} + \sqrt{\sigma_t^2 - \sigma_{t-2}^2}\epsilon \\ &= \ldots = x_0 + \sigma_t \epsilon, \qquad \text{where } \sigma_0 \approx 0. \end{aligned} \tag{42}$$

The continuous version of SMLD

From

$$x_t = x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2}\epsilon, \tag{43}$$

we have

$$\begin{aligned} \Delta x_t &= \sqrt{\frac{\sigma_t^2 - \sigma_{t-1}^2}{\Delta t}}\sqrt{\Delta t}\epsilon \\ &= \sqrt{\frac{\sigma_t^2 - \sigma_{t-1}^2}{\Delta t}}\Delta B_t. \end{aligned} \tag{44}$$

Further, let $\Delta t \to 0$, we have

$$dx = \sqrt{\frac{d\sigma_t^2}{dt}}dB_t. \tag{45}$$

The continuous version of DDPM

Recall that

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon. \tag{46}$$

Similarly

$$x_t \approx (1 - \frac{1}{2}\beta_t)x_{t-1} + \sqrt{\beta_t}\epsilon, \tag{47}$$

hence, we have

$$\begin{aligned} \Delta x_t &= -\frac{1}{2}\tilde{\beta}_t x_{t-1}\Delta t + \sqrt{\tilde{\beta}_t}\sqrt{\Delta t}\epsilon, \quad \text{where } \tilde{\beta}_t = \frac{\beta_i}{\Delta t} \\ &= -\frac{1}{2}\tilde{\beta}_t x_{t-1}\Delta t + \sqrt{\tilde{\beta}_t}\Delta B_t. \end{aligned} \tag{48}$$

Further, let $\Delta t \to 0$, we finally obtain

$$dx = -\frac{1}{2}\tilde{\beta}_t x dt + \sqrt{\tilde{\beta}_t}dB_t. \tag{49}$$

VE & VP

Summarizing the form of continuous DDPM and SMLD, we summarize the adding noise process as

$$dx = f(x,t)dt + g(t)dB_t, \tag{50}$$

where the corresponding distribution $p(x,t)$ satisfying the following Fokker-Planck equation

$$\frac{\partial p(x,t)}{\partial t} + \nabla \cdot (f(x,t)p(x,t)) - \frac{1}{2}g^2(t)\triangle p(x,t) = 0. \tag{51}$$

Training DSM object

$$J_{DSM} = \mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{p(x_0)}\mathbb{E}_{p(x_t|x_0)}\left[\|s_\theta(x_t,t) - \nabla_{x_t}\log p(x_t|x_0)\|^2\right]. \tag{52}$$

VE & VP adding noise

To training the Denoising Score Match loss J_{DSM} above, the key is to add noise by the transition kernel

$$p(\tilde{x}|x) \equiv p(x(t)|x(0)) = N(x(t); \tilde{\alpha}_t x(0), \tilde{\sigma}_t^2), \tag{53}$$

from which we can obtain the noised data

$$x(t) = \tilde{\alpha}_t x(0) + \tilde{\sigma}_t \epsilon, \quad \epsilon \sim N(0, 1). \tag{54}$$

The question becomes: Given VE & VP

$$dx = \sqrt{\frac{d[\sigma^2]}{dt}} dB_t, \quad dx = -\frac{1}{2}\tilde{\beta}_t x dt + \sqrt{\tilde{\beta}_t} dB_t. \tag{55}$$

how to derive $p(x(t)|x(0))$, i.e., $p(\tilde{x}|x)$ for adding noise?

VE & VP Transition kernel

The summarizing form of VE & VP is

$$dx = h(t)xdt + g(t)dB_t, \tag{56}$$

where $h(t) = 0$ for VE and $h(t) = -\frac{1}{2}\tilde{\beta}(t)$ for VP.

Lemma

Let $\mu(t) = \mathbb{E}[x(t)]$ and $\Sigma(t) = \mathbb{E}\left[(x - \mu(t))(x - \mu(t))^T\right]$, we have the following formula

$$\frac{d\mu(t)}{dt} = h(t)\mu(t), \quad \frac{d\Sigma(t)}{dt} = 2h(t)\Sigma(t) + g^2(t)I. \tag{57}$$

The proof of the Lemma can be found in a stochastic differential equation textbook such as Applied Stochastic Differential Equations, Särkkä and Solin, 2019.

VE & VP Transition kernel

Lemma

For VE, the transition kernel is given by the following formula

$$P(x(t)|x(0)) = N(x(t); x(0), \sigma^2(t) - \sigma^2(0)). \tag{58}$$

For VE,

$$\tilde{\alpha}_t \equiv 1, \quad \tilde{\sigma}_t^2 \equiv \sigma^2(t) - \sigma^2(0) \approx \sigma^2(t). \tag{59}$$

Lemma

For VP, the transition kernel is given the following formula

$$p(x(t)|x(0)) = N\left(x(t); x(0)e^{-\frac{1}{2}\int_0^t \tilde{\beta}(s)ds}, 1 - e^{-\int_0^t \tilde{\beta}(s)ds}\right). \tag{60}$$

For VP,

$$\tilde{\alpha}_t \equiv e^{-\frac{1}{2}\int_0^t \tilde{\beta}(s)ds}, \quad \tilde{\sigma}_t^2 \equiv 1 - e^{-\int_0^t \tilde{\beta}(s)ds}. \tag{61}$$

Proof of VE transition kernel

Proof.

For VE, by a simple calculation we have

$$\frac{d\mu(t)}{dt} = 0, \quad \frac{d\Sigma(t)}{dt} = \frac{d[\sigma^2(t)]}{dt}, \tag{62}$$

Hence, it is easy to obtain the following form

$$\mu(t) = \mu(0) = x(0), \quad \Sigma(t) = \sigma^2(t) - \sigma^2(0). \tag{63}$$

Here we have used the fact

$$\mu(0) = x(0), \quad \Sigma(0) = 0 \tag{64}$$

as $x(0)$ is taken as a constant, not a random variable. And we obtain

$$P(x(t)|x(0)) = N(x(t); x(0), \sigma^2(t) - \sigma^2(0)). \tag{65}$$

VP Proof: Differential Equations

Proof.

For VP, we have the differential equations:

$$\frac{d\mu(t)}{dt} = -\frac{1}{2}\beta(t)\mu(t) \tag{66}$$

$$\frac{d\Sigma(t)}{dt} = -\beta(t)\Sigma(t) + \beta(t) \tag{67}$$

We solve these using integrating factors. □

Proof (Cont).

For the mean equation, multiply by $e^{\int_0^t \frac{1}{2}\beta(s)ds}$:

$$\frac{d\mu(t)}{dt}e^{\int_0^t \frac{1}{2}\beta(s)ds} + \frac{1}{2}\beta(t)\mu(t)e^{\int_0^t \frac{1}{2}\beta(s)ds} = 0 \tag{68}$$

For the variance equation, multiply by $e^{\int_0^t \beta(s)ds}$:

$$\frac{d\Sigma(t)}{dt}e^{\int_0^t \beta(s)ds} + \beta(t)\Sigma(t)e^{\int_0^t \beta(s)ds} = \beta(t)e^{\int_0^t \beta(s)ds} \tag{69}$$

VP Proof: Solution

Proof (Cont).

This gives us:

$$\frac{d}{dt}(\mu(t)e^{\int_0^t \frac{1}{2}\beta(s)ds}) = 0 \tag{70}$$

$$\frac{d}{dt}(\Sigma(t)e^{\int_0^t \beta(s)ds}) = \frac{d}{dt}e^{\int_0^t \beta(s)ds} \tag{71}$$

Solving with initial conditions $\mu(0) = x(0)$, $\Sigma(0) = 0$:

$$p(x(t)|x(0)) = N\left(x(t); x(0)e^{-\frac{1}{2}\int_0^t \beta(s)ds}, 1 - e^{-\int_0^t \beta(s)ds}\right) \tag{72}$$

Revisit adding noise

The **equivalence** of the adding noise

1. From the stochastic process, denote the variable x_t, we have

$$dx_t = f(x,t)dt + g(t)dB_t, \tag{73}$$

with corresponding discrete form (Euler-Maruyama scheme)

$$x_{t+\Delta t} = x_t + f(x_t,t)\Delta t + g(t)\sqrt{\Delta t}\epsilon, \quad \epsilon \sim N(0,1). \tag{74}$$

2. From transition kernel perspective,

$$p(x_t) = \int_{x_0} p(x_0)p(x_t|x_0)dx_0, \tag{75}$$

where

$$p(x_t|x_0) = N(x_t;\mu_t,\Sigma_t) = N(x_t;\tilde{\alpha}_t x_0, \tilde{\sigma}_t^2) = \frac{1}{C}e^{-\frac{\|x_t-\mu_t\|^2}{2\Sigma_t}}. \tag{76}$$

Training recipe

The training for VE follows the following process (VP is in a similar way)

1. Random choose one data $x \sim p_{data}$;
2. Random sample $t \sim U[0,1]$;
3. Random sample a white noise $\epsilon \sim N(0,1)$;
4. Adding noise
$$x_t = \tilde{\alpha}_t x + \tilde{\sigma}_t \epsilon, \tag{77}$$
where the mean value $\tilde{\alpha}_t x = x(0)$ and standard deviation $\tilde{\sigma}_t = \sqrt{\sigma^2(t) - \sigma^2(0)} \approx \sigma(t)$ in VE referring to (58). VP is similar with the mean and standard deviation from its transition kernel (60);
5. Compute the Loss by summation the following norm over x, t and ϵ
$$\|s_\theta(x_t, t) - \epsilon/\tilde{\sigma}_t\|. \tag{78}$$

Inference: Reverse denoising process

The reverse denoising process is given by

$$dx = (f(x,t) - g^2 \nabla_x \log p(x,t))dt + g(t)dB_t, \tag{79}$$

or

$$dx = (f(x,t) - \frac{1}{2} g^2 \nabla_x \log p(x,t))dt, \tag{80}$$

or

$$dx = (f(x,t) - \frac{3}{2} g^2 \nabla_x \log p(x,t))dt + \sqrt{2} g(t)dB_t, \tag{81}$$

$$\cdots \tag{82}$$

Due to obeying the same Fokker-Planck equation for $p(x,t)$

$$\frac{\partial p(x,t)}{\partial t} + \nabla \cdot (f(x,t)p(x,t)) - \frac{1}{2} g^2(t) \triangle p(x,t) = 0. \tag{83}$$

Reverse Process Proof: Setup

We prove the first reverse sampling form (79). The Fokker-Planck equation of the forward process (50) is:

$$\frac{\partial p}{\partial t} + \nabla \cdot (f(x,t)p) - \frac{1}{2}g^2 \triangle p = 0. \tag{84}$$

Let $t = T - \tau$, we have:

$$\frac{\partial p}{\partial \tau} - \nabla \cdot (f(x, T-\tau)p) + g^2 \triangle p - \frac{1}{2}g^2 \triangle p = 0. \tag{85}$$

Reverse Process Proof: Derivation

Proof (Cont).

By $\nabla \cdot \nabla = \triangle$ and $\nabla \log p = \nabla p / p$:

$$\frac{\partial p}{\partial \tau} - \nabla \cdot \left(\left(f(x, T-\tau) - g^2 \nabla \log p \right) p \right) - \frac{1}{2} g^2 \triangle p = 0. \tag{86}$$

Hence, we obtain the corresponding SDE form:

$$dx = -(f(x, T-\tau) - g^2 \nabla \log p) d\tau + g dB_\tau \tag{87}$$

By substitution $\tau = T - t$:

$$dx = (f(x, t) - g^2 \nabla \log p) dt + g dB_t \tag{88}$$

Inference recipe

The inference for VE follows the following process (VP is in a similar way)

1. Random generate a noise at time $t = 1$ as $x(1) \sim N(0, \sigma_{max}^2)$;
2. Integrate the reverse sampling equation

$$dx = \left(f(x,t) - g^2(t)s_\theta(x,t)\right)dt + g(t)dB_t \tag{89}$$

   or

$$dx = (f - \frac{1}{2}g^2 s_\theta)dt \tag{90}$$

   over the time span $[0, 1]$ to get $x(0)$.
3. Corrector process: at each internal value $x(t)$, we can relax the value $x(t)$ by a corrector, which takes the Langevin MCMC process.

The Corrector: revisit Langevin MCMC

The Langevin diffusion process is given by

$$x_i = x_{i-1} + \frac{\Delta t}{2}\nabla_x \log p(x_{i-1}) + \sqrt{\Delta t}\epsilon, \quad \epsilon \sim N(0,1). \tag{91}$$

We can prove $\pi(x)$ of the obtained data points $\{x_i\}_{i=1}^N$ converges to $p(x)$.

Proof.

The continuous form of the scheme is:

$$dx = \frac{1}{2}\nabla \log p(x)dt + dB_t. \tag{92}$$

The corresponding Fokker-Planck equation is:

$$\frac{\partial \pi(x,t)}{\partial t} + \nabla \cdot (\frac{1}{2}\nabla \log p(x)\pi(x,t)) - \frac{1}{2}\triangle \pi(x,t) = 0. \tag{93}$$

With sufficient time evolution, the distribution converges to a stable distribution where:

Proof (Cont).

Hence we have:

$$\nabla \cdot (\pi \nabla \log p - \nabla \pi) = 0, \quad \forall x. \tag{95}$$

Therefore:

$$\nabla \log p = \nabla \log \pi \tag{96}$$

given that $\pi > 0$ almost everywhere. Hence:

$$\pi^*(x) = p(x) \tag{97}$$

where π^* is the stable distribution of $\pi(x, t)$.

Outline

Additional Important Papers

The field of diffusion models has evolved rapidly with many important contributions. Here we highlight several key papers that have shaped the current understanding and practice:

- **DDPM** - The foundational work on denoising diffusion probabilistic models
- **EDM** - Elucidating design choices for better performance
- **DPM-Solver** - Fast ODE solvers for efficient sampling
- **Consistency Models** - One-step generation with diffusion quality

These papers represent significant advances in making diffusion models more practical and efficient.

Outline

DDPM: Goal & Setup

Goal. Learn a generative model $p_\theta(x_0)$ by reversing a noisy diffusion.

Latents. Introduce $x_{1:T}$ with same dimensionality as data x_0.

Reverse (sampling) chain:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \quad p(x_T) = \mathcal{N}(0, I),$$

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\, \Sigma_\theta(x_t, t)\big).$$

Forward (fixed) diffusion:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \quad q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I\big).$$

With $\alpha_t := 1 - \beta_t$, $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$:

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\, x_0,\, (1 - \bar{\alpha}_t) I\big).$$

DDPM: Training Objective (Variational Bound)

Optimize the NLL via a reduced-variance ELBO:

$$\mathbb{E}[-\log p_\theta(x_0)] \;\le\; \underbrace{\mathbb{E}\big[\mathrm{KL}(q(x_T|x_0)\,\|\,p(x_T))\big]}_{L_T} + \sum_{t=2}^{T} \underbrace{\mathbb{E}\big[\mathrm{KL}(q(x_{t-1}|x_t,x_0)\,\|\,p_\theta(x_{t-1}|x_t))\big]}_{L_{t-1}} - \underbrace{\mathbb{E}[\log p_\theta}_{L}$$

Here $q(x_{t-1}\,|\,x_t, x_0)$ is Gaussian in closed form, so each KL has a tractable expression.

Data scaling. Images scaled to $[-1, 1]$; $p_\theta(x_0\,|\,x_1)$ is a discretized Gaussian decoder for likelihood evaluation.

DDPM: ϵ-Prediction Parameterization

Write the forward noising as

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Parameterize the reverse mean via a network $\varepsilon_\theta(x_t, t)$:

$$\mu_\theta(x_t, t) \;=\; \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon_\theta(x_t, t) \right).$$

Sampling step (with fixed σ_t such as $\sigma_t^2 = \beta_t$ or $\tilde{\beta}_t$):

$$x_{t-1} \;=\; \mu_\theta(x_t, t) \;+\; \sigma_t\, z, \quad z \sim \mathcal{N}(0, I).$$

This connects DDPM to denoising score matching and (annealed) Langevin-like sampling.

DDPM: Simplified Loss & Algorithms

Simplified objective ("L_{simple}"):

$$\mathcal{L}_{\text{simple}}(\theta) \;=\; \mathbb{E}_{t\sim\mathcal{U}[1,T],\,x_0,\,\varepsilon}\Big\|\,\varepsilon - \varepsilon_\theta\big(\sqrt{\bar\alpha_t}\,x_0 + \sqrt{1-\bar\alpha_t}\,\varepsilon,\; t\big)\Big\|_2^2$$

Training (sketch).

1. Sample $x_0 \sim q(x_0)$, $t \sim \mathrm{Unif}\{1,\dots,T\}$, $\varepsilon \sim \mathcal{N}(0, I)$.
2. Form x_t via the forward formula; take a GD step on $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$.

Sampling (sketch).

1. $x_T \sim \mathcal{N}(0, I)$.
2. For $t = T, \dots, 1$: compute $\mu_\theta(x_t, t)$, add noise $\sigma_t z$ if $t > 1$ to get x_{t-1}.

Empirically, L_{simple} gives best sample quality; ELBO training gives better likelihoods.

DDPM: Schedules, Architecture, & Practice

Noise schedule. Typical: $T=1000$, linear β_t from 10^{-4} to 2×10^{-2} (on $[-1,1]$ data); small β_t keeps forward and reverse both Gaussian-friendly.

Network. U-Net backbone (PixelCNN++-style), group norm, sinusoidal time embedding, self-attention at medium resolution (e.g., 16×16).

Tips.

- Fixed Σ_θ (e.g., $\sigma_t^2 = \beta_t$ or $\tilde{\beta}_t$) stabilizes training.
- Dropout (small) on CIFAR-10 helps; EMA on params (e.g., 0.9999).
- Evaluate with FID/IS; likelihood via discretized decoder.

DDPM: Intuitions & Connections

Progressive generation. Coarse structure emerges early (large t), details refine as $t\downarrow$.

Score-based view. ε_θ learns noise (scaled score), making the sampler resemble annealed Langevin dynamics.

Autoregressive link. The ELBO can be seen as progressive decoding with a generalized "bit ordering."

Why DDPM?

- High sample quality; stable training; flexible architectures.
- Trade-off: exact likelihoods lag some likelihood-based models, but samples excel.

Outline

Big Picture

- Goal: separate **design choices** (sampling, schedules, preconditioning, training) to simplify and speed up diffusion models.
- Key outcomes:
  - **Faster sampling** (tens of NFEs) with strong FID.
  - Modular choices: ODE view, schedule $\sigma(t)$, scaling $s(t)$, time steps $\{t_i\}$, preconditioning, loss weighting.
- Works as a **drop-in** improvement over prior models (VP/VE/DDIM/ADM).

Common ODE Framework

Probability flow ODE with schedule $\sigma(t)$ and (optional) scaling $s(t)$:

$$\frac{d\boldsymbol{x}}{dt} = \left(\frac{\dot{\sigma}}{\sigma} + \frac{\dot{s}}{s}\right)\boldsymbol{x} \; - \; \frac{\dot{\sigma}\, s}{\sigma}\, \nabla_{\boldsymbol{x}} \log p\left(\tfrac{\boldsymbol{x}}{s}; \sigma\right)$$

Using denoising score matching,

$$\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}; \sigma) = \frac{D(\boldsymbol{x}; \sigma) - \boldsymbol{x}}{\sigma^2},$$

so the ODE is solved numerically by evaluating a **denoiser** $D_\theta(\boldsymbol{x}; \sigma)$.

EDM choice: set $\sigma(t) = t,\ s(t) = 1 \Rightarrow$ straight(er) trajectories, stable integration.

Denoising Score Matching (Training Target)

Given clean $\boldsymbol{y} \sim p_{\text{data}}$, noise $\boldsymbol{n} \sim \mathcal{N}(0, \sigma^2 I)$,

$$D(\boldsymbol{y} + \boldsymbol{n}; \sigma) \;=\; \arg\min_{D} \mathbb{E} \,\|\, D(\boldsymbol{y} + \boldsymbol{n}; \sigma) - \boldsymbol{y} \,\|_2^2,$$

and

$$\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}; \sigma) = \frac{D(\boldsymbol{x}; \sigma) - \boldsymbol{x}}{\sigma^2}.$$

Train D_θ over a range of σ to match the optimal denoiser; use it at test time to integrate the ODE.

Network Preconditioning

Represent denoiser with **preconditioned** skip connection:

$$D_\theta(\boldsymbol{x};\sigma) = c_{\text{skip}}(\sigma)\,\boldsymbol{x} \;+\; c_{\text{out}}(\sigma)\,F_\theta\big(c_{\text{in}}(\sigma)\,\boldsymbol{x};\; c_{\text{noise}}(\sigma)\big),$$

Design $c_{\text{in}}, c_{\text{out}}, c_{\text{skip}}$ so that:

- inputs/targets have **nearly unit variance** across σ,
- network errors are not **amplified** at large σ (robust gradients),
- F_θ smoothly interpolates between predicting signal vs. noise.

This stabilizes training and unlocks better loss scheduling.

Loss Weighting & Noise-Level Sampling

With the preconditioned form, the effective per-sample weight scales with $c_{\text{out}}(\sigma)^2$.

$$\textbf{Set} \quad \lambda(\sigma) = \frac{1}{c_{\text{out}}(\sigma)^2} \quad \Rightarrow \quad \text{balanced contributions over } \sigma.$$

Sample noise levels σ from a **log-normal** (or similar) that prioritizes the *perceptually relevant* mid-σ range:

$$\ln \sigma \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2).$$

Result: large FID gains across datasets when combined with preconditioning.

Augmentation Regularization (Non-Leaky)

- Apply standard geometric augmentations to training images *before* adding noise.
- Feed augmentation parameters as conditioning to F_θ (*non-leaky*); set to zero at inference.
- Helps small/medium datasets reduce overfitting; further FID improvements.

Headline Results & Takeaways

- **Sampling:** Heun(2) + $\sigma(t)=t$ + ρ-scheduled steps $\Rightarrow$ big NFE cuts at equal FID.
- **Training:** preconditioning + balanced loss + noise-level prior $\Rightarrow$ strong, robust models.
- **Stochasticity:** beneficial on harder data; unnecessary with strong training on simpler data.
- **Outcomes:** new SOTA-level FIDs on CIFAR-10 / ImageNet-64 with **much faster** sampling.

Outline

DPM-Solver: Motivation

- Diffusion models (DPMs) produce strong samples but are slow: **hundreds–thousands** of NFE.
- Sampling can be recast as solving a **probability flow ODE**: no stochasticity $\Rightarrow$ larger steps possible.
- **DPM-Solver** exploits the *semi-linear* structure of the diffusion ODE to deliver high-quality samples in **10–20 NFE**.
- Key idea: compute the *linear part* analytically; only approximate a special, exponentially weighted integral of the network.

DPM-Solver: From SDE to ODE

Forward SDE (VP-type example):

$$d\boldsymbol{x}_t = f(t)\boldsymbol{x}_t\,dt + g(t)\,d\boldsymbol{w}_t, \quad f(t) = \tfrac{d}{dt}\log\alpha_t, \quad g^2(t) = \tfrac{d}{dt}\sigma_t^2 - 2\tfrac{d}{dt}\log\alpha_t \cdot \sigma_t^2.$$

Probability flow ODE with learned noise predictor $\varepsilon_\theta(\boldsymbol{x}, t)$:

$$\frac{d\boldsymbol{x}_t}{dt} = f(t)\boldsymbol{x}_t + \frac{g^2(t)}{2\sigma_t}\varepsilon_\theta(\boldsymbol{x}_t, t), \quad \boldsymbol{x}_T \sim \mathcal{N}(0, \tilde{\sigma}^2\boldsymbol{I}).$$

Semi-linear form: linear in $\boldsymbol{x}_t$ plus a nonlinear NN term.

DPM-Solver: Exact Solution via Variation of Constants

Given state $\boldsymbol{x}_s$ at $s > t$, the exact solution of the diffusion ODE is

$$\boldsymbol{x}_t = e^{\int_t^s f(\tau)\,d\tau}\,\boldsymbol{x}_s + \int_t^s e^{\int_t^\tau f(r)\,dr}\,\frac{g^2(\tau)}{2\sigma_\tau}\,\varepsilon_\theta(\boldsymbol{x}_\tau, \tau)\,d\tau.$$

Introduce half-logSNR $\lambda_t := \log(\alpha_t/\sigma_t)$ (strictly decreasing in t). Using change of variables,

$$\boxed{\boldsymbol{x}_t = \frac{\alpha_t}{\alpha_s}\,\boldsymbol{x}_s \;-\; \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda}\,\hat{\varepsilon}_\theta(\hat{\boldsymbol{x}}_\lambda, \lambda)\,d\lambda}$$

Only the **exponentially weighted integral** of the network remains to be approximated.

DPM-Solver: Why This Helps

- **Analytic linear term** removes its discretization error (a major source of instability for large steps).
- The remaining integral matches the structure tackled by **exponential integrators**.
- Work entirely in λ (half-logSNR) space: step selection $\{\lambda_i\}$ becomes **noise-schedule invariant**.

DPM-Solver-1 (First Order)

Let time grid $t_0 = T > t_1 > \cdots > t_M = 0$, with $\lambda_i = \lambda(t_i)$, $h_i = \lambda_i - \lambda_{i-1}$.

$$\boxed{\tilde{\boldsymbol{x}}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{\boldsymbol{x}}_{t_{i-1}} - \sigma_{t_i}\left(e^{h_i} - 1\right) \varepsilon_\theta(\tilde{\boldsymbol{x}}_{t_{i-1}}, t_{i-1})}$$

Fact: This update is algebraically identical to **DDIM** in λ-parameterization.

DPM-Solver-2 (Second Order)

Midpoint in λ: $s_i = t_\lambda\left(\frac{\lambda_{i-1}+\lambda_i}{2}\right)$.

$$\boldsymbol{u}_i = \frac{\alpha_{s_i}}{\alpha_{t_{i-1}}}\tilde{\boldsymbol{x}}_{t_{i-1}} - \sigma_{s_i}\left(e^{h_i/2}-1\right)\varepsilon_\theta(\tilde{\boldsymbol{x}}_{t_{i-1}}, t_{i-1}),$$
$$\tilde{\boldsymbol{x}}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\tilde{\boldsymbol{x}}_{t_{i-1}} - \sigma_{t_i}\left(e^{h_i}-1\right)\varepsilon_\theta(\boldsymbol{u}_i, s_i).$$

Second order accuracy in λ with 2 network evaluations per step.

DPM-Solver-3 (Third Order)

Two intermediate points $s_{2i-1} = t_\lambda(\lambda_{i-1} + \frac{1}{3}h_i)$, $s_{2i} = t_\lambda(\lambda_{i-1} + \frac{2}{3}h_i)$.

Evaluate ε_θ at t_{i-1}, s_{2i-1}, s_{2i} and form finite-difference corrections D_{2i-1}, D_{2i}.

$$\tilde{\boldsymbol{x}}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\tilde{\boldsymbol{x}}_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\varepsilon_\theta(\tilde{\boldsymbol{x}}_{t_{i-1}}, t_{i-1}) - \sigma_{t_i}\frac{2}{3}\left(\frac{e^{h_i} - 1}{h_i} - 1\right)D_{2i}$$

Third order accuracy in λ with 3 evaluations per step; converges in very few steps.

DPM-Solver: Choosing Step Sizes

Uniform in λ: split $[\lambda_T, \lambda_0]$ into M equal parts:

$$\lambda_i = \lambda_T + \frac{i}{M}(\lambda_0 - \lambda_T).$$

Adaptive (for larger NFE budgets): combine orders (1,2,3) with local error control in λ.

Few-step recipe (e.g., total NFE ≤ 20*):* use as many **DPM-Solver-3** steps as possible; if leftover evaluations remain, finish with a single DPM-Solver-2 (or -1) step.

DPM-Solver: Discrete-Time DPMs

Models trained at integer steps $n = 0, \ldots, N-1$ with $\tilde{\varepsilon}_\theta(\boldsymbol{x}_n, n)$ can be used by defining a continuous-time wrapper:

$$\varepsilon_\theta(\boldsymbol{x}, t) := \tilde{\varepsilon}_\theta\left(\boldsymbol{x}, \frac{(N-1)t}{T}\right),$$

leveraging smooth time embeddings. Then apply DPM-Solver in continuous t/λ directly.

Quality & Efficiency (Takeaways)

- **10–20 NFE** typically suffice for high-fidelity samples across standard benchmarks.
- Outperforms generic RK solvers at the same order due to **analytic linear part** and λ-**space** integration.
- Works with classifier guidance and both continuous/discrete DPMs *without retraining*.

DPM-Solver: Implementation Tips

- Precompute $\alpha_t, \sigma_t, \lambda_t$ and (if available) closed-form $t_\lambda(\cdot)$ for your noise schedule.
- Use float32 for networks but consider float64 for λ-grid arithmetic if extremely few steps are used.
- Start with uniform-λ grids; move to adaptive control only if pushing NFE below ≈ 10.
- For guidance, share predictor calls between unconditional and conditional branches when possible.

Outline

Consistency Models: Motivation

- Diffusion models (DMs) achieve SOTA quality but require many denoising steps $\Rightarrow$ slow sampling.
- **Goal:** Keep DM perks (quality, zero-shot editing, compute/quality trade-off) while enabling **fast 1-step** generation.
- **Idea:** Learn a map that sends any point on a Probability-Flow (PF) ODE trajectory back to its origin (data).

PF ODE (from SDE):

$$\mathrm{d}x_t = \left(\mu(x_t, t) - \tfrac{1}{2}\sigma(t)^2 \nabla \log p_t(x_t)\right) \mathrm{d}t \quad \Rightarrow \quad \frac{\mathrm{d}x_t}{\mathrm{d}t} = -\,t\, s_\phi(x_t, t)$$

(where $s_\phi \approx \nabla \log p_t$ under EDM-style settings)

Consistency Function & Model (Definition)

Consistency function

Given a PF ODE trajectory $\{x_t\}_{t\in[\varepsilon,T]}$, define

$$f : (x_t, t) \mapsto x_\varepsilon, \quad \text{with self-consistency } f(x_t, t) = f(x_{t'}, t') \text{ if on same trajectory.}$$

Consistency model

A neural estimator $f_\theta(x, t)$ of f trained to enforce self-consistency.

- **Boundary condition:** $f_\theta(x, \varepsilon) = x$ (identity at $t = \varepsilon$).
- Practical parameterization with skip:

$$f_\theta(x, t) = c_{\text{skip}}(t)\, x + c_{\text{out}}(t)\, F_\theta(x, t), \;\; c_{\text{skip}}(\varepsilon) = 1, \;\; c_{\text{out}}(\varepsilon) = 0.$$

Sampling: One-step and Few-step

One-step generation

$$x_T \sim \mathcal{N}(0, T^2 I), \quad x_\varepsilon = f_\theta(x_T, T).$$

Few-step (trade quality for compute): alternate denoise & noise injection

$x \leftarrow f_\theta(x_T, T)$, then for $\tau_1 > \cdots > \tau_{N-1}$: $\hat{x}_{\tau_n} \leftarrow x + \sqrt{\tau_n^2 - \varepsilon^2}\, z$, $z \sim \mathcal{N}(0, I)$; $x \leftarrow f_\theta(\hat{x}_{\tau_n}, \tau_n)$.

Time points $\{\tau_n\}$ can be greedily tuned (e.g., by FID unimodality assumption).

Training I: Consistency Distillation (CD)

Setting: Pretrained diffusion score model $s_\phi(x, t)$; discretize $[\varepsilon, T]$ into $\{t_1 = \varepsilon < \cdots < t_N = T\}$.

One-step ODE update (e.g., Heun/Euler) from $x_{t_{n+1}}$ to $\hat{x}^\phi_{t_n}$:

$$\hat{x}^\phi_{t_n} \triangleq x_{t_{n+1}} + (t_n - t_{n+1})\,\Phi(x_{t_{n+1}}, t_{n+1}; \phi).$$

CD loss: encourage equal outputs for adjacent points on same trajectory

$$\mathcal{L}^{(N)}_{\mathrm{CD}}(\theta, \theta^-; \phi) = \mathbb{E}\big[\lambda(t_n)\, d\Big(f_\theta(x_{t_{n+1}}, t_{n+1}),\ f_{\theta^-}(\hat{x}^\phi_{t_n}, t_n)\Big)\big]$$

with target EMA $\theta^- \leftarrow \mu\,\theta^- + (1-\mu)\,\theta$.

Notes: LPIPS metric and Heun solver with moderate N often work best; higher-order solvers tighten error bounds.

Training II: Consistency Training (CT, No Teacher)

Goal: Train f_θ *without* any pretrained diffusion.

$$\mathcal{L}_{\mathrm{CT}}^{(N)}(\theta, \theta^-) = \mathbb{E}_{x\sim p_{\mathrm{data}},\, z\sim\mathcal{N}(0,I),\, n}\left[\lambda(t_n)\, d(f_\theta(x + t_{n+1}z, t_{n+1}),\ f_{\theta^-}(x + t_n z, t_n))\right].$$

- Connection: as $N\to\infty$ (small Δt) with Euler, $\mathcal{L}_{\mathrm{CT}}$ matches $\mathcal{L}_{\mathrm{CD}}$ up to $o(\Delta t)$.
- Practice: progressively increase N and the EMA decay μ during training for faster convergence then higher final quality.

Consistency Models: Why it Works (Sketches)

- **Self-consistency** gives a functional fixed-point: f_θ constant along PF-ODE trajectories.
- **Boundary** $f_\theta(\cdot, \varepsilon) = \mathrm{Id}$ prevents trivial collapse.
- **CD theory:** If $\mathcal{L}_{\mathrm{CD}} = 0$ and solver local error is $\mathcal{O}(\Delta t^{p+1})$, then

$$\sup_{n,x} \|f_\theta(x, t_n) - f(x, t_n; \phi)\| = \mathcal{O}(\Delta t^p).$$

- **CT theory:** replaces teacher scores via unbiased score identities; continuous-time variant avoids bias (needs fwd-mode AD).

Zero-shot Editing via Few-step Loop

- **Denoising** across noise levels (feed x_t for any t).
- **Inpainting / Colorization / Super-resolution / Stroke-guided editing**: modify parts / constraints between denoise & re-noise steps, akin to diffusion inversion.
- **Latent interpolations:** interpolate in x_T space and map with f_θ.

Consistency Models: Quality & Speed (Selected Numbers)

- **CD (one-step)**: CIFAR-10 FID ≈ 3.55; ImageNet 64×64 FID ≈ 6.20.
- **CD (two-step)**: CIFAR-10 ≈ 2.93; ImageNet $64 \times 64 \approx 4.70$.
- **CT (direct, no teacher)**: surpasses many one-step non-adversarial baselines on CIFAR-10; improves with two-step sampling.

Zero-shot editing demonstrated on LSUN Bedroom/Cat 256^2 (colorization, SR, inpainting, stroke-guided).

Practical Tips

- Use EDM-style parameterization for $c_{\text{skip}}(t), c_{\text{out}}(t)$ and borrow U-Net backbones from diffusion.
- For CD: LPIPS metric and Heun solver with N around 18 (dataset-dependent) often best.
- For CT: schedule N and EMA μ to grow over training; continuous-time CT is elegant but needs forward-mode AD.
- Few-step sampling times $\{\tau_n\}$ can be greedily tuned (e.g., ternary search) to optimize FID.

Takeaways

- **Consistency models** provide fast **one-step** generation while enabling **few-step** trade-offs.
- Two training routes: **CD** (distill diffusion) and **CT** (train in isolation).
- Maintain diffusion-style **zero-shot editing** abilities with simpler sampling.

Outline

Questions

If you are interested in the questions, please feel free to write an email to me (hpp@bimsa.cn) with your comments.

1. Please prove that the forms (79)-(81) are equivalent with same marginal distribution $p(x,t)$ given the same initial condition;
2. Please prove that the score function $s(x,t)$ satisfying the following formula

$$\frac{\partial s}{\partial t} = \nabla(-\nabla \cdot f - \langle f, s\rangle + \frac{1}{2}g^2\|s\|^2 + \frac{1}{2}g^2\langle \nabla, s\rangle); \tag{98}$$

3. Why ISM loss is not commonly used like DSM loss in the diffusion model nowadays? Please make your comments.

Welcome inputs

Any comments?

Welcome your inputs!

Outline

Probability Path and Coupling

- Let $\{p_t\}_{t\in[0,1]}$ be a prescribed path of marginals on $\mathbb{R}^d$ with p_0 simple and p_1 the data.
- A *coupling* induces random trajectories $t \mapsto x_t$ with instantaneous velocity $\dot{x}_t = \frac{\mathrm{d}}{\mathrm{d}t}x_t$ (may be stochastic given x_t).
- Goal: learn a *deterministic* vector field $v : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ such that the ODE

$$\frac{\mathrm{d}}{\mathrm{d}t}x_t = v(x_t, t), \qquad x_0 \sim p_0,$$

yields marginals $x_t \sim p_t$ for all t.

Remark

We can depict the flow matching and diffusion through the probability transition v.s. the trajectories. And also, we can connect the Neural ODE with the Flow matching.

Strong and Weak Continuity

From the Fokker–Planck equation, for $\frac{\mathrm{d}x}{\mathrm{d}t} = v(x,t)$, we have

$$\partial_t p_t(x) + \nabla\cdot\big(p_t(x)\, v(x,t)\big) = 0.$$

The corresponding weak form of above strong form is

Weak form (for any test functions $\varphi \in C_c^\infty(\mathbb{R}^d)$)

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{\mathbb{R}^d} \varphi(x)\, p_t(x)\, \mathrm{d}x \;=\; \int_{\mathbb{R}^d} \nabla\varphi(x)\cdot v(x,t)\, p_t(x)\, \mathrm{d}x,$$

i.e.,

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\mathbb{E}[\varphi(x_t)] = \mathbb{E}[\nabla\varphi(x_t)\cdot v(x,t)].$$

Weak Form from Random Trajectories

Let x_t be the coupled trajectory (not necessarily deterministic). For any test function $\phi \in C_c^\infty(\mathbb{R}^d)$, we have

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\mathbb{E}[\varphi(x_t)] = \mathbb{E}[\nabla\varphi(x_t)\cdot\dot{x}_t],$$

by the expectation of the derivative of the test function.
Expanding the expectation into integrals at a fixed t:

$$\begin{aligned}
\mathbb{E}[\nabla\varphi(x_t)\cdot\dot{x}_t] &= \iint_{\mathbb{R}^d\times\mathbb{R}^d} \nabla\varphi(x)\cdot\dot{x}\;\mu_t(\mathrm{d}x,\,\mathrm{d}\dot{x}) \\
&= \int_{\mathbb{R}^d} \nabla\varphi(x)\cdot\Big(\int \dot{x}\,p(\dot{x}\,|\,x,t)\;\mathrm{d}\dot{x}\Big)\,p_t(x)\;\mathrm{d}x \\
&= \int_{\mathbb{R}^d} \nabla\varphi(x)\cdot v^\star(x,t)\,p_t(x)\;\mathrm{d}x = \mathbb{E}[\nabla\varphi(x_t)\cdot v^\star(x,t)].
\end{aligned}$$

where $v^\star(x,t) \equiv \mathbb{E}[\dot{x}_t \mid x_t = x,\ t] = \int_{\mathbb{R}^d} \dot{x}\,p(\dot{x}\,|\,x,t)\;\mathrm{d}\dot{x}$. Thus the random path induces the weak identity with $v = v^\star$.

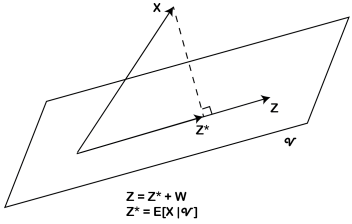
Theoretical Support: Conditional Expectation

Theorem

Let X be an integrable random variable. Then for each σ-algebra $\mathcal{V}$ and $Z \in \mathcal{V}$, $Z^* = \mathbb{E}(X|\mathcal{V})$ solves the least square problem

$$argmin_{Z \in \mathcal{V}} \|Z - X\|,$$

where $\|Z\| = \left(\int Z^2 dP\right)^{\frac{1}{2}}$.



Hence we have $L = \mathbb{E}\|v_\theta(x_t, t) - \dot{x}_t\|^2$.

Practical Training Recipe (Per-Iteration)

1. Sample $t \sim w(t)$ (typically uniform on $[0,1]$).
2. Sample $(x_t, \dot{x}_t)$ from the chosen path coupling.
3. Regress $\dot{x}_t$ on (x_t, t) via the MSE:

$$\mathcal{L} = \left\| v_\theta(x_t, t) - \dot{x}_t \right\|^2.$$

4. Update θ by SGD; no density estimation, no reverse simulation is needed during training.

Remark

Different path/coupling choices change $v^\star$ and the irreducible variance $\mathbb{E}\|\dot{x}_t - v^\star\|^2$, affecting sample efficiency and optimization, but the optimal target remains the conditional mean.

Key Takeaways

- The correct notion of "matching the path" is the **weak continuity** identity for all test functions.
- The only deterministic field depending on (x, t) that preserves this identity is the **conditional mean velocity**

$$\boxed{v^\star(x,t) = \mathbb{E}[\dot{x}_t \mid x_t = x,\ t]}.$$

- The **flow-matching MSE** is precisely the L^2 projection selecting $v^\star$:

$$v_\theta \xrightarrow[\text{minimize}]{\mathbb{E}\|v_\theta(x_t,t)-\dot{x}_t\|^2} v^\star.$$

Outline

Theorem (Fokker–Planck / Forward Kolmogorov)

Let $(X_t)_{t\geq 0} \in \mathbb{R}^N$ solve the Itô SDE

$$dX_t = \mu(X_t, t)\, dt + \boldsymbol{\sigma}(X_t, t)\, dW_t,$$

where $\mu : \mathbb{R}^N \times [0,\infty) \to \mathbb{R}^N$ is the drift, $\boldsymbol{\sigma} : \mathbb{R}^N \times [0,\infty) \to \mathbb{R}^{N\times M}$ the diffusion, and W_t an M-dimensional standard Wiener process. Define the diffusion tensor

$$D(x,t) := \frac{1}{2}\, \boldsymbol{\sigma}(x,t)\, \boldsymbol{\sigma}(x,t)^{\top} \in \mathbb{R}^{N\times N}.$$

Then the (smooth) density $p(x,t)$ satisfies the Fokker–Planck equation

$$\partial_t p(x,t) = -\sum_{i=1}^{N} \frac{\partial}{\partial x_i}\big[\mu_i(x,t)\, p(x,t)\big] \;+\; \sum_{i,j=1}^{N} \frac{\partial^2}{\partial x_i\, \partial x_j}\big[D_{ij}(x,t)\, p(x,t)\big]$$

Isotropic diffusion: $\boldsymbol{\sigma}(x,t) = \sigma(t)\,\mathbf{I}_N$

When $\boldsymbol{\sigma}(x,t) = \sigma(t)\,\mathbf{I}_N$, the diffusion tensor becomes

$$D(x,t) = \tfrac{1}{2}\,\sigma(t)^2\,\mathbf{I}_N, \qquad D_{ij}(x,t) = \tfrac{1}{2}\,\sigma(t)^2\,\delta_{ij}.$$

Hence the Fokker–Planck equation simplifies to the drift–diffusion form

$$\boxed{\partial_t p(x,t) = -\nabla\cdot\big(\mu(x,t)\,p(x,t)\big) \;+\; \frac{\sigma(t)^2}{2}\,\Delta p(x,t)}$$

where $\nabla\cdot(\mu p) = \sum_{i=1}^{N} \partial_{x_i}\big(\mu_i p\big)$ and $\Delta = \sum_{i=1}^{N} \partial_{x_i}^2$ is the Laplacian.

Mathematical connection to diffusion models

Theoretical link: For a forward SDE $\mathrm{d}\mathbf{x} = f(\mathbf{x}, t)\,\mathrm{d}t + g(t)\,\mathrm{d}B_t$, the probability flow ODE is:

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbf{x} = f(\mathbf{x}, t) - \tfrac{1}{2}g^2(t)\,\nabla_{\mathbf{x}} \log p(\mathbf{x}, t). \tag{99}$$

Key mathematical insight: When Flow Matching uses Gaussian paths that match the SDE's marginal distributions p_t, the learned velocity field $\mathbf{v}_\theta$ approximates the probability flow drift:

$$\mathbf{v}_\theta(\mathbf{x}, t) \approx f(\mathbf{x}, t) - \tfrac{1}{2}g^2(t)\,\nabla_{\mathbf{x}} \log p(\mathbf{x}, t) \tag{100}$$

Crucial advantage: Flow Matching achieves this approximation *without* ever computing the score $\nabla_{\mathbf{x}} \log p(\mathbf{x}, t)$ during training!

Practical implications: This connection allows Flow Matching to inherit the theoretical guarantees of diffusion models while avoiding their computational bottlenecks.

Mathematical derivation: probability flow ODE

Setup: Forward SDE $\mathrm{d}\mathbf{x} = f(\mathbf{x}, t)\,\mathrm{d}t + g(t)\,\mathrm{d}B_t$ with density $p(\mathbf{x}, t)$ solving the Fokker-Planck equation:

$$\partial_t p = -\nabla \cdot (fp) + \tfrac{1}{2} g^2 \triangle p. \tag{101}$$

Key insight: A deterministic ODE $\dot{\mathbf{x}} = \mathbf{v}(\mathbf{x}, t)$ transports densities via the continuity equation:

$$\partial_t p + \nabla \cdot (p\,\mathbf{v}) = 0. \tag{102}$$

Mathematical derivation: Equating the two equations:

$$-\nabla \cdot (p\,\mathbf{v}) = -\nabla \cdot (fp) + \tfrac{1}{2} g^2 \triangle p \tag{103}$$

$$\Rightarrow\ \mathbf{v} = f - \tfrac{1}{2} g^2 \frac{\nabla p}{p} \ =\ f - \tfrac{1}{2} g^2 \nabla \log p. \tag{104}$$

Result: The probability flow ODE drift is $\mathbf{v} = f - \frac{1}{2} g^2 \nabla \log p$.
Connection to Flow Matching: When Flow Matching learns $\mathbf{v}_\theta$, it approximates this probability flow drift without computing the score $\nabla \log p$.

Questions

From

1. Score model $s_\theta(x_t) \to s_{\theta^*}(x_t) \equiv \mathbb{E}[\nabla \log p(x_t|x_0) \mid x_t]$
2. X prediction model (denoising model, $x0$ model) $x_\theta(x_t) \to x_{\theta^*}(x_t) \equiv \mathbb{E}[x_0 \mid x_t]$;
3. ϵ prediction model $\epsilon_\theta(x_t) \to \epsilon_{\theta^*}(x_t) \equiv \mathbb{E}[\epsilon \mid x_t]$.

and

$$v^\star(x_t) = \mathbb{E}[\dot{x}_t \mid x_t]$$

, what is the relationship between optimal $v_\theta(x, t)$ and the above models? Hints:

$$v_{\theta^*}(x_t) = \mathbb{E}[\dot{x}_t \mid x_t] = \mathbb{E}[\dot{\alpha}(t)x_t + \dot{\sigma}(t)\epsilon \mid x_t]. \tag{105}$$

and

$$s(x_t) = \mathbb{E}_x[\nabla_{x_t} \log p(x_t|x_0)|x_t] = \mathbb{E}_x[-\frac{x_t - \alpha x_0}{\sigma^2}|x_t] = -\frac{x_t - \alpha \mathbb{E}[x_0|x_t]}{\sigma^2}. \tag{106}$$

Outline

Gaussian conditional paths: mathematical derivation

Assumption: Gaussian conditional paths

$$q_t(\mathbf{x} \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x};\ \boldsymbol{\mu}_t(\mathbf{x}_0),\, \sigma_t^2 \mathbf{I}\big), \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \boldsymbol{\epsilon}. \tag{107}$$

Mathematical derivation: Taking the time derivative of the path:

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbf{x}_t = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t\, \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} = \frac{\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0)}{\sigma_t}. \tag{108}$$

Conditional expectation: Since $\boldsymbol{\epsilon}$ is independent of $\mathbf{x}_t$ given $\mathbf{x}_0$, we have:

$$\boxed{v = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \frac{\dot{\sigma}_t}{\sigma_t}\Big(\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0)\Big).} \tag{109}$$

Mathematical analysis of path choices

1. **Linear bridge** (optimal transport):
   - $\boldsymbol{\mu}_t(\mathbf{x}_0) = (1-t)\,\mathbf{x}_0$, $\sigma_t = t\,\sigma_{\max}$
   - **Mathematical property**: Minimizes Wasserstein-2 distance
   - **Velocity**: $v = \frac{\sigma_{\max}}{t}(\mathbf{x} - \mathbf{x}_0)$
   - **Advantage**: Direct connection to optimal transport theory
2. **VP-like** (DDPM-inspired):
   - $\boldsymbol{\mu}_t(\mathbf{x}_0) = \alpha(t)\,\mathbf{x}_0$, $\sigma_t = \sqrt{1-\alpha^2(t)}$
   - **Mathematical property**: Preserves energy structure
   - **Velocity**: $v = \dot{\alpha}(t)\mathbf{x}_0 + \frac{\dot{\sigma_t}}{\sigma_t}(\mathbf{x} - \alpha(t)\mathbf{x}_0)$
   - **Advantage**: Compatible with existing diffusion model schedules
3. **VE-like** (SMLD-inspired):
   - $\boldsymbol{\mu}_t(\mathbf{x}_0) = \mathbf{x}_0$, $\sigma_t = \sigma(t)$ increasing
   - **Mathematical property**: Preserves data structure
   - **Velocity**: $v = \frac{\dot{\sigma}(t)}{\sigma(t)}(\mathbf{x} - \mathbf{x}_0)$
   - **Advantage**: Simple form, good for high-dimensional data

Key insight: All choices provide *closed-form* $\mathbf{u}_t$ via the boxed formula!

Derivation: VE/VP target forms

VE-like. $\boldsymbol{\mu}_t(\mathbf{x}_0) = \mathbf{x}_0$, $\sigma_t = \sigma(t)$. Then

$$v = 0 + \frac{\dot{\sigma}_t}{\sigma_t}(\mathbf{x} - \mathbf{x}_0). \tag{110}$$

VP-like. $\boldsymbol{\mu}_t(\mathbf{x}_0) = \alpha(t)\,\mathbf{x}_0$, $\sigma_t = \sqrt{1-\alpha^2(t)}$. Using $\mathbf{u}_t = \dot{\boldsymbol{\mu}}_t + (\dot{\sigma}_t/\sigma_t)(\mathbf{x} - \boldsymbol{\mu}_t)$, we get

$$v = \dot{\alpha}\,\mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t}\mathbf{x} - \frac{\dot{\sigma}_t}{\sigma_t}\alpha\,\mathbf{x}_0 \tag{111}$$

$$= \frac{\dot{\sigma}_t}{\sigma_t}\mathbf{x} + \left(\dot{\alpha} - \alpha\frac{\dot{\sigma}_t}{\sigma_t}\right)\mathbf{x}_0. \tag{112}$$

No explicit form for $\dot{\sigma}_t/\sigma_t$ is required to train; schedules determine it numerically.
Mathematical insight: These derivations show how different path choices lead to different velocity field structures, all computable analytically.

Outline

Flow Matching: mathematical training objective

Theoretical foundation: Given conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0)$, the optimal velocity field satisfies:

$$v^\star(x,t) = \mathbb{E}[\dot{x}_t \mid x_t = x,\ t] \tag{113}$$

Training procedure:

1. Sample $t \sim \mathcal{U}[0,1]$, $\mathbf{x}_0 \sim p_{\text{data}}$, $\mathbf{x}_t \sim q_t(\cdot \mid \mathbf{x}_0)$
2. Compute analytic target $\mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0)$ from chosen path
3. Minimize regression loss

Mathematical objective:

$$J_{\text{FM}} = \mathbb{E}\big[\, \| \mathbf{v}_\theta(\mathbf{x}_t, t) - v \|^2 \,\big]. \tag{114}$$

Key mathematical advantage: No score estimation needed; all targets are analytic!

Computational benefits:

- Stable gradients (no exploding/vanishing)
- Parallelizable across time steps
- No special architectural requirements

Mathematical foundation of inference

Theoretical basis: The learned velocity field $\mathbf{v}_\theta(\mathbf{x}, t)$ defines a deterministic ODE:

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t), \quad t \in [0, 1] \tag{115}$$

Mathematical properties:

- **Existence**: Under Lipschitz conditions, solutions exist and are unique
- **Consistency**: The ODE preserves the continuity equation
- **Stability**: Deterministic integration avoids accumulation of stochastic errors

Generation process:

1. Initialize $\mathbf{x}(1) \sim p_1$ (e.g., $\mathcal{N}(0, \mathbf{I})$)
2. Integrate ODE backwards: $\frac{\mathrm{d}}{\mathrm{d}t}\mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t)$, $t : 1 \to 0$
3. Use standard ODE solvers (Euler, RK4, Dormand-Prince, etc.)

Mathematical advantages:

- Deterministic sampling (reproducible)
- Fast with large solver steps
- No stochastic correctors needed

Outline

Conditional Flow Matching

Extension to conditional generation: Instead of unconditional paths, we can define conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0, c)$ where c represents conditioning information (e.g., class labels, text prompts).

Mathematical formulation: The conditional velocity field becomes:

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0, c) := \mathbb{E}\big[\tfrac{\mathrm{d}}{\mathrm{d}t} \mathbf{x}_t \;\big|\; \mathbf{x}_t = \mathbf{x},\, \mathbf{x}_0,\, c \big] \tag{116}$$

Training objective:

$$J_{\mathrm{CFM}} = \mathbb{E}\big[\| \mathbf{v}_\theta(\mathbf{x}_t, t, c) - v \|^2 \big] \tag{117}$$

Applications:

- Class-conditional generation
- Text-to-image synthesis
- Image-to-image translation
- Style transfer

Advantage: Maintains the same computational efficiency as unconditional Flow Matching while enabling controlled generation.

Rectified Flow and Optimal Transport

Rectified Flow: A special case of Flow Matching that uses straight-line paths:

$$\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1, \quad v = \mathbf{x}_1 - \mathbf{x}_0 \tag{118}$$

Mathematical properties:

- **Optimal transport**: Minimizes Wasserstein-2 distance
- **Simplicity**: Constant velocity field along each path
- **Efficiency**: Fastest convergence in many cases

Connection to optimal transport: Rectified Flow recovers the optimal transport map between p_0 and p_1 when the data is well-structured.

Practical advantages:

- Fewer function evaluations during sampling
- More stable training
- Better sample quality in many domains

Flow Matching in Latent Spaces

Motivation: Instead of working directly in pixel space, Flow Matching can be applied in learned latent representations.

Mathematical setup: Given an encoder $E : \mathcal{X} \to \mathcal{Z}$ and decoder $D : \mathcal{Z} \to \mathcal{X}$:

- Learn flow in latent space: $\mathbf{v}_\theta : \mathcal{Z} \times [0, 1] \to \mathcal{Z}$
- Generate latent codes: $\mathbf{z}_0 = \mathbf{z}_1 + \int_0^1 \mathbf{v}_\theta(\mathbf{z}_t, t)dt$
- Decode to images: $\mathbf{x} = D(\mathbf{z}_0)$

Advantages:

- **Computational efficiency**: Lower-dimensional space
- **Sample quality**: Better inductive biases in latent space
- **Flexibility**: Can use different architectures for encoder/decoder

Examples: Latent Diffusion Models (LDMs), Stable Diffusion

Outline

MeanFlow: Motivation

Goal: Achieve single-step (1-NFE) generation while retaining high fidelity and stability.
Problem with standard Flow Matching: Multi-step ODE integration is computationally expensive and can accumulate errors.
Key insight: Learn a *ground-truth average velocity field* that summarizes an entire time interval.
Mathematical formulation: For any pair $r < t$ and point $\mathbf{z}_t$ along the flow path:

$$\overline{\mathbf{u}}(\mathbf{z}_t, r, t) := \frac{1}{t-r} \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau)\, \mathrm{d}\tau. \tag{119}$$

Properties:

- $\overline{\mathbf{u}}$ is an **underlying field** induced by $\mathbf{v}$ (no networks yet)
- As $r \to t$, $\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \to \mathbf{v}(\mathbf{z}_t, t)$
- Composition over intervals holds: one big step equals two smaller consecutive steps

Definition: Average Velocity

For any pair $r < t$ and point $\mathbf{z}_t$ along the flow path:

$$\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;\coloneqq\; \frac{1}{t-r} \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau)\, \mathrm{d}\tau. \tag{120}$$

- $\overline{\mathbf{u}}$ is an **underlying field** induced by $\mathbf{v}$ (no networks yet).
- As $r \to t$, $\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \to \mathbf{v}(\mathbf{z}_t, t)$.
- Composition over intervals holds: one big step equals two smaller consecutive steps (additivity of integrals).

Preliminaries: Matrix function derivations and Jacobian-vector product

- Matrix function derivations
- Einsten notation and Einstein summation convention
- Jacobian-vector product

See wikipedia matrix calculus for more details.

MeanFlow Identity (core relation)

Starting from Eq. (119):

$$(t-r)\,\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;=\; \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau)\;\mathrm{d}\tau. \tag{121}$$

Differentiate w.r.t. t (treating r as constant) and use the fundamental theorem of calculus:

$$\boxed{\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;=\; \mathbf{v}(\mathbf{z}_t, t) \;-\; (t-r)\,\frac{d}{dt}\,\overline{\mathbf{u}}(\mathbf{z}_t, r, t)} \tag{122}$$

where the total derivative expands to

$$\frac{d}{dt}\,\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;=\; \underbrace{\mathbf{v}(\mathbf{z}_t, t)}_{\frac{d\mathbf{z}_t}{dt}}\cdot\partial_{\mathbf{z}}\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;+\; \partial_t\overline{\mathbf{u}}(\mathbf{z}_t, r, t). \tag{123}$$

Interpretation: Eq. (122) links instantaneous and average velocities *without* introducing networks.

Mathematical significance: This identity enables learning average velocity fields that can be used for single-step generation.

Trainable objective

Parameterize a network $\overline{\mathbf{u}}_\theta(\mathbf{z}, r, t)$. Use the MeanFlow identity to form a target (replace $\mathbf{v}$ by sample-conditional $\mathbf{v}_t$):

$$\text{Target:} \quad \overline{\mathbf{u}}_{\text{tgt}} = \mathbf{v}_t - (t-r)\big(\underbrace{\mathbf{v}_t \cdot \partial_{\mathbf{z}}\overline{\mathbf{u}}_\theta}_{\text{JVP}} + \partial_t \overline{\mathbf{u}}_\theta\big). \tag{124}$$

$$\mathcal{L}(\theta) = \mathbb{E}\left\| \overline{\mathbf{u}}_\theta(\mathbf{z}_t, r, t) - \mathrm{sg}(\overline{\mathbf{u}}_{\text{tgt}}) \right\|_2^2. \tag{125}$$

Key components:

- $\mathrm{sg}(\cdot)$: stop-gradient; avoids higher-order differentiation
- JVP (Jacobian-vector product) computed efficiently via autodiff
- If $r = t$, the correction term vanishes $\Rightarrow$ reduces to standard Flow Matching

Mathematical insight: This objective learns to predict average velocities over time intervals, enabling single-step generation.

Pseudocode (training)

MeanFlow training algorithm:

1. Sample $t, r \sim \text{LogitNormal}$; set $t \leftarrow \max(r, t)$, $r \leftarrow \min(r, t)$
2. Sample $\mathbf{x} \sim p_{\text{data}}$, $\epsilon \sim \mathcal{N}(0, I)$; set $\mathbf{z}_t = (1-t)\mathbf{x} + t\epsilon$, $\mathbf{v}_t = \epsilon - \mathbf{x}$
3. Compute $\big(\overline{\mathbf{u}}_\theta,\ \frac{d}{dt}\overline{\mathbf{u}}_\theta\big) = \mathrm{JVP}\big(\overline{\mathbf{u}}_\theta(\cdot, r, t),\ (\mathbf{v}_t, 0, 1)\big)$
4. Set $\overline{\mathbf{u}}_{\text{tgt}} \leftarrow \mathbf{v}_t - (t-r)\,\frac{d}{dt}\overline{\mathbf{u}}_\theta$
5. Minimize $\|\overline{\mathbf{u}}_\theta(\mathbf{z}_t, r, t) - \mathrm{sg}(\overline{\mathbf{u}}_{\text{tgt}})\|_2^2$ (optionally with adaptive weights)

Key innovations:

- **LogitNormal sampling**: Ensures proper time interval distribution
- **JVP computation**: Efficient higher-order derivatives
- **Stop-gradient**: Prevents training instability

Sampling (1 step or few steps)

Replace the time integral by the learned average velocity:

$$\mathbf{z}_r \;=\; \mathbf{z}_t - (t-r)\,\overline{\mathbf{u}}_\theta(\mathbf{z}_t, r, t). \tag{126}$$

1-NFE sampling: take $(r,t) = (0,1)$, $\mathbf{z}_1 = \epsilon$,

$$\boxed{\mathbf{x} \equiv \mathbf{z}_0 = \epsilon - \overline{\mathbf{u}}_\theta(\epsilon, 0, 1)}\,. \tag{127}$$

Mathematical significance: This single-step generation maintains the same theoretical guarantees as multi-step Flow Matching.

Practical advantages:

- **Speed**: Single function evaluation
- **Stability**: No accumulation of integration errors
- **Flexibility**: Can still use few-step variants by chaining intervals

CFG inside the field (still 1-NFE)

Define a CFG velocity field

$$\mathbf{v}_{\text{cfg}}(\mathbf{z}_t, t \mid c) \;=\; \omega\, \mathbf{v}(\mathbf{z}_t, t \mid c) + (1-\omega)\, \mathbf{v}(\mathbf{z}_t, t). \tag{128}$$

Key insight: CFG can be applied directly to the average velocity field:

$$\overline{\mathbf{u}}_{\text{cfg}}(\mathbf{z}_t, r, t \mid c) \;=\; \omega\, \overline{\mathbf{u}}(\mathbf{z}_t, r, t \mid c) + (1-\omega)\, \overline{\mathbf{u}}(\mathbf{z}_t, r, t). \tag{129}$$

Mathematical property: This maintains the 1-NFE property while enabling conditional generation.
Practical significance: MeanFlow can achieve both single-step generation and strong conditioning, combining the best of both worlds.

Outline

Key Takeaways: Flow Matching

Mathematical foundations:

- Flow Matching learns velocity fields that transport probability distributions via the continuity equation
- Conditional paths provide analytic supervision without score estimation
- The method connects to optimal transport theory and diffusion models

Computational advantages:

- No score estimation during training
- Deterministic sampling with flexible step sizes
- Stable gradients and efficient training

Practical benefits:

- Fewer function evaluations than diffusion models
- Better sample quality in many domains
- Natural extension to conditional generation

MeanFlow extension: Enables single-step generation while maintaining theoretical guarantees.

Future Directions and Open Problems

Theoretical developments:

- **Path optimization**: Finding optimal conditional paths for specific data distributions
- **Convergence analysis**: Understanding when Flow Matching achieves optimal transport
- **Error bounds**: Quantifying approximation errors in finite-sample settings

Practical improvements:

- **Architecture design**: Specialized networks for different data modalities
- **Training strategies**: Adaptive loss weighting and curriculum learning
- **Sampling efficiency**: Better ODE solvers and step size selection

Applications:

- **Scientific computing**: Molecular dynamics, fluid simulation
- **Creative AI**: Music, video, and 3D content generation
- **Data augmentation**: Synthetic data for training other models

Outline

Key References

Core Flow Matching papers:

- Lipman et al., *Flow Matching for Generative Modeling* (2022)
- Geng et al., *MeanFlow: A Flow Matching Method for One-Step Generation* (2025)
- Liu et al., *Learning to Generate and Transfer Data with Rectified Flow* (2022)
- Song et al., *Score-based generative modeling through stochastic differential equations* (2020)
- Albergo et al., *stochastic interpolants: a unifying framework for flows and diffusions* (2023)

Implementation resources:

- Flow Matching Guide and Code (GitHub)
- Open source libraries: diffusers
- Tutorial notebooks and examples

北京雁栖湖应用数学研究院
Beijing Institute of Mathematical Sciences and Applications

Welcome inputs

Any comments?

Welcome your inputs!

Outline

Applications Overview

Generative models (diffusion and flow matching) enable powerful applications in scientific discovery:

- Molecular & Materials Design: Generating novel compounds with desired properties
- Protein Structure & Complex Generation: Designing proteins and protein complexes
- Property-Conditioned Generation: Sampling structures with specific functional properties

Key challenges: Structural validity, prior knowledge integration, constraint handling

Key Challenges in AI for Science

Scientific problems present unique challenges for machine learning:

1. **Data scarcity**: Limited experimental data, expensive to collect
2. **Physical constraints**: Solutions must obey physical laws
3. **Interpretability**: Scientific understanding requires explainable models
4. **Uncertainty quantification**: Scientific predictions need confidence bounds
5. **Multi-scale phenomena**: From quantum to macroscopic scales

Outline

Molecular and Materials Generation with Generative Models

Generative models (diffusion and flow matching) enable novel molecular and materials design:

- **Molecular generation**: Design drug-like molecules with desired properties
- **Materials discovery**: Generate crystal structures with target electronic/mechanical properties
- **Property-conditioned sampling**: Generate structures with specific functional properties

Key considerations:

- Structural validity (chemical rules, bond constraints)
- Prior knowledge integration (force fields, symmetry)
- Constraint handling (physical constraints, domain knowledge)

Outline

Molecular Representation Learning

Converting molecules to machine-readable formats:

- **SMILES strings**: Text-based molecular representation
- **Graph neural networks**: Molecular graphs with atoms as nodes, bonds as edges
- **3D coordinates**: Spatial arrangement of atoms
- **Descriptors**: Hand-crafted molecular features

Graph Neural Networks for Molecules

For a molecular graph $G = (V, E)$ with atoms $v_i \in V$ and bonds $e_{ij} \in E$:
Message passing:

$$h_v^{(l+1)} = \text{UPDATE}\left(h_v^{(l)}, \text{AGGREGATE}\left(\{h_u^{(l)} : u \in \mathcal{N}(v)\}\right)\right) \tag{130}$$

Property prediction:

$$y = \text{READOUT}(\{h_v^{(L)} : v \in V\}) \tag{131}$$

where L is the number of message passing layers.

Applications in Drug Discovery

- **Property prediction**: Solubility, toxicity, binding affinity
- **Molecular generation**: Designing new drug candidates
- **Retrosynthesis**: Planning synthetic routes
- **Protein-ligand binding**: Predicting drug-target interactions

Key datasets:

- QM9: 134k small organic molecules; GEOM-Drugs
- ChEMBL: Large-scale bioactivity data
- PDB: Protein structures; AlphaFoldDB

Outline

The Protein Folding Problem

Proteins fold from linear amino acid sequences to 3D structures:

$$\text{Sequence: ACDEFGHIKLMNPQRSTVWY} \rightarrow \text{3D Structure} \tag{132}$$

Key challenges:

- Exponential number of possible conformations
- Complex energy landscapes
- Multiple timescales (nanoseconds to seconds)

AlphaFold and Structure Prediction

AlphaFold approach:

1. **Multiple sequence alignment**: Evolutionary information
2. **Attention mechanisms**: Long-range interactions
3. **Structure module**: 3D coordinate prediction
4. **Confidence estimation**: Reliability of predictions

Key innovations:

- End-to-end differentiable architecture
- Integration of evolutionary and physical constraints
- Uncertainty quantification

architecture

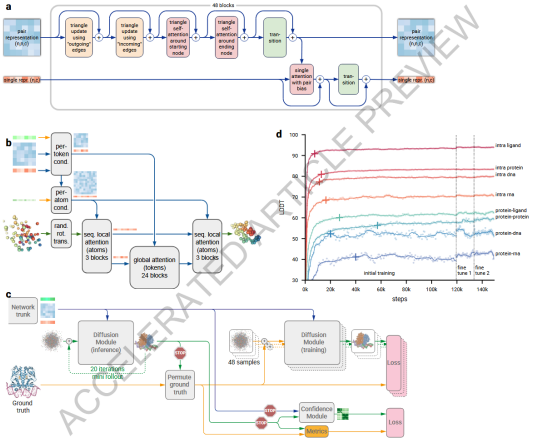


Figure: The architecture of AlphaFold 3.

Impact of AlphaFold

- **Structural biology revolution**: Millions of protein structures predicted
- **Drug discovery**: Better understanding of drug targets
- **Protein design**: Engineering proteins with desired functions
- **Evolutionary biology**: Understanding protein evolution

Current limitations:

- Limited accuracy for disordered regions
- Challenges with protein complexes
- Difficulty with membrane proteins

Materials Property Prediction

Predicting material properties from composition and structure:

- **Electronic properties**: Band gap, conductivity, magnetic properties
- **Mechanical properties**: Elastic modulus, strength, hardness
- **Thermal properties**: Heat capacity, thermal conductivity
- **Optical properties**: Refractive index, absorption spectra

Crystal Structure Representation

Periodic systems: Representing crystal structures with periodic boundary conditions.
Graph representations:

- **Crystal graphs**: Atoms as nodes, bonds as edges
- **Periodic embeddings**: Handling translational symmetry
- **Multi-scale features**: From atomic to macroscopic properties

Deep learning approaches:

- **CGCNN**: Crystal Graph Convolutional Neural Networks
- **SchNet**: Quantum-chemical property prediction
- **MEGNet**: Materials property prediction

High-Throughput Materials Discovery

Computational screening:

1. Generate candidate materials
2. Predict properties using ML models
3. Filter promising candidates
4. Experimental validation

Key databases:

- **Materials Project**: 150k+ materials with computed properties
- **OQMD**: Open Quantum Materials Database
- **AFLOW**: Automatic Flow for Materials Discovery

Outline

Methodological Reflections: Towards Scientific Workflows

Generative models for science must go beyond black-box generation. Key considerations:

1. **Interpretability**: Understanding what models learn and how they make decisions
2. **Controllability**: Property-conditioned generation, constraint handling, prior knowledge integration
3. **Data Efficiency**: Learning from limited experimental data, leveraging physical priors
4. **Multimodal Integration**: Combining different data types (sequences, structures, properties)

Goal: Models as tools for scientific discovery, not pure black-box generators.

Interpretability: Understanding Model Behavior

- **Score functions** reveal learned data geometry and energy landscapes
- **Flow trajectories** show generation paths and intermediate states
- **Attention mechanisms** (in transformers) highlight important structural features
- **Uncertainty quantification** provides confidence estimates for predictions

Example: In protein folding, attention patterns reveal which sequence regions drive structure formation.

Controllability: Property-Conditioned Generation

- **Conditional generation**: Sample structures with specific properties (binding affinity, stability, etc.)
- **Constraint handling**: Enforce physical constraints (bond lengths, angles, symmetry)
- **Prior knowledge**: Incorporate domain knowledge (chemical rules, biological constraints)
- **Guided sampling**: Use classifier guidance or energy-based models for targeted generation

Example: Generate drug-like molecules with desired binding properties while respecting chemical validity.

Data Efficiency: Learning from Limited Data

- **Physical priors**: Incorporate known physics (force fields, conservation laws)
- **Transfer learning**: Pre-train on large datasets, fine-tune on domain-specific data
- **Active learning**: Selectively query expensive experiments
- **Data augmentation**: Leverage symmetries and invariances (rotations, translations)

Example: Pre-train on AlphaFoldDB, fine-tune on experimental structures for specific protein families.

Multimodal Integration: Combining Data Types

- **Sequence + Structure**: Joint modeling of amino acid sequences and 3D coordinates
- **Structure + Properties**: Link geometric features to functional properties
- **Text + Structure**: Natural language descriptions of molecular function
- **Experimental + Computational**: Combine simulation and experimental data

Example: AlphaFold 3 integrates sequences, structures, and experimental data for complex prediction.

Outline

Outline

Concept & Setup

Goal. Predict full-atom 3D structures from an amino-acid sequence using a *general-purpose* Transformer trained with flow matching.

Design stance.

- No MSA, no explicit pair representations, no triangular updates, no equivariant blocks.
- Treat sequence as a conditioning signal (like a text prompt) and generate all-atom coordinates.

Flow-Matching Scheme

Continuous-time generation.

$$\mathrm{d}\boldsymbol{x}_t = \boldsymbol{v}_\theta(\boldsymbol{x}_t, t)\,\mathrm{d}t, \quad t \in [0,1]$$

Map a tractable prior p_0 (Gaussian) to data p_D via a learned velocity field.

Training interpolant (rectified flow).

$$\boldsymbol{x}_t = t\,\boldsymbol{x} + (1-t)\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}), \quad \boldsymbol{x} \sim p_D$$

Target velocity: $\boldsymbol{v}_t = \boldsymbol{x} - \boldsymbol{\epsilon}$.
Loss: $\mathbb{E}\big[\|\boldsymbol{v}_\theta(\boldsymbol{x}_t, t) - \boldsymbol{v}_t\|_2^2\big]$.

Sampling

Stochastic flow sampling. Initialize $\boldsymbol{x}_0 \sim \mathcal{N}(0, \mathbf{I})$, integrate to $t = 1$ with Euler–Maruyama:

$$\mathrm{d}\boldsymbol{x}_t = \boldsymbol{v}_\theta(\boldsymbol{x}_t, s, t)\,\mathrm{d}t + \tfrac{1}{2} w(t)\,\boldsymbol{s}_\theta(\boldsymbol{x}_t, s, t)\,\mathrm{d}t + \sqrt{\tau\, w(t)}\,\mathrm{d}\bar{\boldsymbol{W}}_t,$$

where $\boldsymbol{s}_\theta = \dfrac{t\,\boldsymbol{v}_\theta - \boldsymbol{x}_t}{1-t}$ and $w(t) = \dfrac{2(1-t)}{t+\eta}$.

Role of τ. Controls diversity/accuracy trade-off for ensembles.

Folding with Flow Matching

Conditioning. Given sequence s and N_a heavy atoms:

$$\boldsymbol{x}_t, \boldsymbol{x}, \boldsymbol{\epsilon} \in \mathbb{R}^{N_a \times 3}, \qquad \boldsymbol{v}_\theta = \boldsymbol{v}_\theta(\boldsymbol{x}_t, s, t)$$

Main objective (per-atom average).

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\boldsymbol{x},s,\boldsymbol{\epsilon},t}\left[\frac{1}{N_a}\,\|\boldsymbol{v}_\theta(\boldsymbol{x}_t, s, t) - (\boldsymbol{x} - \boldsymbol{\epsilon})\|_2^2\right]$$

Structural term (LDDT-inspired). One-step Euler estimate $\hat{\boldsymbol{x}}(\boldsymbol{x}_t) = \boldsymbol{x}_t + (1-t)\boldsymbol{v}_\theta(\boldsymbol{x}_t, s, t)$ and penalize local distance errors:

$$\mathcal{L}_{\text{LDDT}} = \mathbb{E}\left[\frac{\sum_{i\neq j} \mathbf{1}(\delta_{ij} < C)\,\sigma\left(\|\delta_{ij} - \hat{\delta}_{ij}^{\,t}\|\right)}{\sum_{i\neq j} \mathbf{1}(\delta_{ij} < C)}\right]$$

Total loss: $\mathcal{L} = \mathcal{L}_{\text{FM}} + \alpha(t)\mathcal{L}_{\text{LDDT}}$.

Timestep resampling. $p(t) = 0.98\,\text{LogisticNormal}(0.8, 1.7) + 0.02\,\mathcal{U}(0,1)$, biased toward $t \to 1$ for fine details.

Architecture Overview

Three modules, one block type.

1. **Atom encoder** (local attention): noisy coords $\boldsymbol{x}_t$ + atom features $\rightarrow$ atom tokens $\boldsymbol{a} \in \mathbb{R}^{N_a \times d_a}$.
2. **Residue trunk** (heavy): group/pool atom tokens to residue tokens $\boldsymbol{r} \in \mathbb{R}^{N_r \times d_a}$, concat with frozen PLM embeddings $e \in \mathbb{R}^{N_r \times d_e}$ from ESM2-3B, process with standard Transformer blocks with adaptive layers.
3. **Atom decoder** (local attention): ungroup residue tokens back to atoms, add skip from encoder, predict velocity $\hat{\boldsymbol{v}}_t$.

Blocks. QK-normalization, SwiGLU FFN, adaptive scale/shift by t; all non-equivariant, no pair/triangle stacks.
Positional encodings. RoPE in trunk; 4D axial RoPE in atom modules (3D reference conformer coords + residue index).

Design Choices vs Prior Work

- **No MSA**, **no explicit pair representation**, **no triangular updates**.
- Learn SO(3) symmetries via **data augmentation** rather than equivariance constraints.
- Hierarchical compute: *fine* (atoms) $\rightarrow$ *coarse* (residues) $\rightarrow$ *fine* (atoms).
- Frozen PLM (ESM2-3B) supplies sequence conditioning; folding model is trained end-to-end with flow matching.

Confidence Module (pLDDT)

What. A separate 4-layer Transformer predicts per-residue pLDDT (0–100).

How.

- Freeze the folding model; feed generated $\hat{\boldsymbol{x}}$ with $t{=}1$ to get final residue tokens.
- Train pLDDT head with cross-entropy over 50 bins.

Use. Correlates with LDDT-Cα and can score structures from other models.

Training Data & Regime

Data sources.

- PDB (cutoff May 2020) $\sim$160k structures.
- AFDB SwissProt subset (pLDDT mean $>$85, std $<$15) $\sim$270k.
- AFESM representatives filtered at pLDDT > 0.8 $\sim$1.9M.
- For 3B model: extended AFESM-E up to $\sim$8.6M distilled structures (max 10 per cluster, mean pLDDT > 80).

Training.

- Pre-train at seq length 256; $\alpha(t) = 1$; AdamW $1e{-4}$; EMA 0.999; large effective batch.
- Finetune on PDB+SwissProt at seq length 512; $\alpha(t) = 1 + 8\,\mathrm{ReLU}(t - 0.5)$ up to 5 near t=1.

Structure and Results

Type	Model	TM-score ↑	GDT-TS ↑	LDDT ↑	LDDT-C_α ↑	RMSD ↓
		CAMEO22				
MSA-based	RoseTTAFold (Baek et al., 2021)	0.780 / 0.860	0.715 / 0.775	0.575 / 0.605	0.798 / 0.827	5.721 / 2.864
	AlphaFlow (Jing et al., 2024a)	0.840 / 0.927	0.808 / 0.853	0.741 / 0.798	0.855 / 0.893	3.846 / 2.122
	AlphaFold2 (Jumper et al., 2021)	0.863 / 0.942	0.844 / 0.903	0.816 / 0.856	0.893 / 0.923	3.578 / 1.857
	RoseTTAFold2 (Baek et al., 2023)	0.864 / 0.947	0.845 / 0.904	0.727 / 0.767	0.893 / 0.926	3.571 / 1.707
PLM-based	ESM3 (Hayes et al., 2025)	0.746 / 0.840	0.694 / 0.758	–	–	–
	ESMDiff (Lu et al., 2024a)	0.754 / 0.847	0.701 / 0.760	–	–	–
	EigenFold (Jing et al., 2023)	0.750 / 0.840	0.710 / 0.790	–	–	–
	OmegaFold (Wu et al., 2022)	0.805 / 0.899	0.767 / 0.844	0.746 / 0.815	0.829 / 0.892	5.294 / 2.622
	ESMFlow (Jing et al., 2024a)	0.818 / 0.893	0.774 / 0.832	0.696 / 0.745	0.827 / 0.867	4.528 / 2.693
	ESMFold (Lin et al., 2023b)	0.853 / 0.933	0.826 / 0.875	0.792 / 0.834	0.871 / 0.906	3.973 / 2.019
Ours	SimpleFold-100M	0.803 / 0.878	0.746 / 0.787	0.721 / 0.752	0.822 / 0.852	4.897 / 2.855
	SimpleFold-360M	0.826 / 0.905	0.782 / 0.841	0.773 / 0.803	0.844 / 0.878	4.775 / 2.681
	SimpleFold-700M	0.829 / 0.915	0.788 / 0.845	0.775 / 0.809	0.850 / 0.886	4.557 / 2.423
	SimpleFold-1.1B	0.833 / 0.924	0.793 / 0.851	0.776 / 0.807	0.850 / 0.883	4.350 / 2.334
	SimpleFold-1.6B	0.835 / 0.916	0.799 / 0.864	0.782 / 0.816	0.853 / 0.889	4.397 / 2.187
	SimpleFold-3B	0.837 / 0.916	0.802 / 0.867	0.773 / 0.802	0.852 / 0.884	4.225 / 2.175
		CASP14				
MSA-based	RoseTTAFold (Baek et al., 2021)	0.654 / 0.678	0.562 / 0.572	0.464 / 0.456	0.705 / 0.723	9.676 / 6.420
	AlphaFlow (Jing et al., 2024a)	0.740 / 0.812	0.661 / 0.711	0.632 / 0.662	0.767 / 0.799	7.091 / 3.949
	RoseTTAFold2 (Baek et al., 2023)	0.802 / 0.881	0.740 / 0.824	0.638 / 0.669	0.824 / 0.869	6.744 / 3.292
	AlphaFold2 (Jumper et al., 2021)	0.845 / 0.907	0.783 / 0.855	0.778 / 0.817	0.856 / 0.897	5.027 / 3.015
PLM-based	ESMDiff (Lu et al., 2024a)	0.521 / 0.499	0.447 / 0.430	–	–	–
	ESM3 (Hayes et al., 2025)	0.534 / 0.567	0.459 / 0.488	–	–	–
	EigenFold (Jing et al., 2023)	0.590 / 0.637	0.539 / 0.575	–	–	–
	ESMFlow (Jing et al., 2024a)	0.627 / 0.679	0.539 / 0.544	0.525 / 0.539	0.669 / 0.730	10.503 / 6.974
	OmegaFold (Wu et al., 2022)	0.693 / 0.773	0.625 / 0.723	0.627 / 0.726	0.715 / 0.824	9.845 / 4.042
	ESMFold (Lin et al., 2023b)	0.701 / 0.792	0.622 / 0.711	0.637 / 0.705	0.725 / 0.802	8.679 / 4.016
Ours	SimpleFold-100M	0.611 / 0.628	0.513 / 0.544	0.537 / 0.549	0.659 / 0.685	11.157 / 8.976
	SimpleFold-360M	0.674 / 0.758	0.585 / 0.654	0.617 / 0.657	0.703 / 0.762	9.382 / 4.828
	SimpleFold-700M	0.680 / 0.767	0.591 / 0.668	0.630 / 0.674	0.714 / 0.763	9.289 / 4.431
	SimpleFold-1.1B	0.697 / 0.796	0.607 / 0.668	0.640 / 0.676	0.723 / 0.758	9.249 / 4.462
	SimpleFold-1.6B	0.712 / 0.801	0.630 / 0.709	0.660 / 0.699	0.741 / 0.798	8.424 / 4.722
	SimpleFold-3B	0.720 / 0.792	0.639 / 0.703	0.666 / 0.709	0.747 / 0.829	7.732 / 3.923

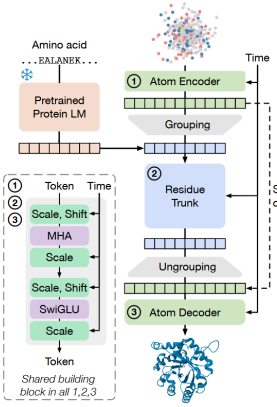


Figure 2 Overview of SimpleFold's architecture built on general-purpose standard Transformer block with adaptive layers. Atom encoder, residue trunk, and atom decoder all share the same general-purposed building block. Our model circumvents the need for pair representations or triangular updates.

Results

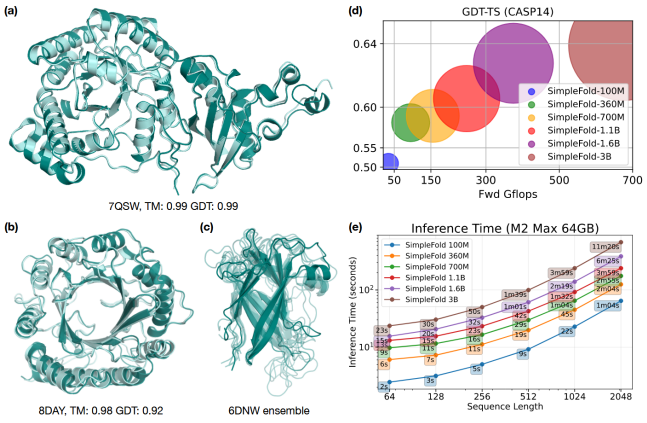


Figure 1 Example predictions of SimpleFold on targets (a) chain A of 7QSW (RubisCO large subunit) and (b) chain A of 8DAY (Dimethylallyltryptophan synthase 1), with ground truth shown in light aqua and prediction in deep teal. (c) Generated ensembles of target chain B of 6NDW (Flagellar hook protein FlgE) with SimpleFold finetuned on MD ensemble data. (d) Performance of SimpleFold on CASP14 with increasing model sizes from 100M to 3B. (e) Inference time of different sizes of SimpleFold on consumer level hardware, i.e., M2 Max 64GB Macbook Pro.

Scalability

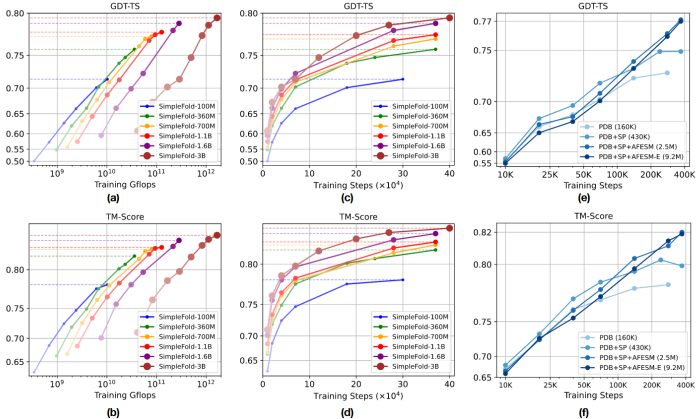


Figure 4 Scaling behavior of SimpleFold. Training Gflops vs. folding performance on GDT-TS and (b) TM-score. Training steps vs. folding performance on (c) GDT-TS and (d) TM-score. How data scale affects the performance (e) GDT-TS and (f) TM-score. All models are benchmarked on CAMEO22.

Outline

AlphaFold 3

See the pdfs.

Outline

UniGenX

See the MS PowerPoint presentation for more details.

Outline

Outline

Outline

Summary

AI for Science represents a paradigm shift in how we approach scientific research:

- **Acceleration**: Faster discovery and development cycles
- **Integration**: Combining data-driven and physics-based approaches
- **Automation**: Reducing manual work in scientific research
- **Insights**: Discovering patterns humans might miss

The Future of AI for Science

- **Democratization**: Making advanced scientific tools accessible to all researchers
- **Interdisciplinary collaboration**: Breaking down silos between scientific fields
- **Scientific method evolution**: How AI changes the scientific process
- **Grand challenges**: Tackling humanity's biggest scientific problems

Questions for Discussion

1. How can we ensure AI for Science maintains scientific rigor and reproducibility?
2. What are the implications of AI for the future of scientific careers?
3. How should we balance AI automation with human scientific intuition?
4. What ethical guidelines should govern AI for Science research?

Welcome inputs

Any comments?

Welcome your inputs!